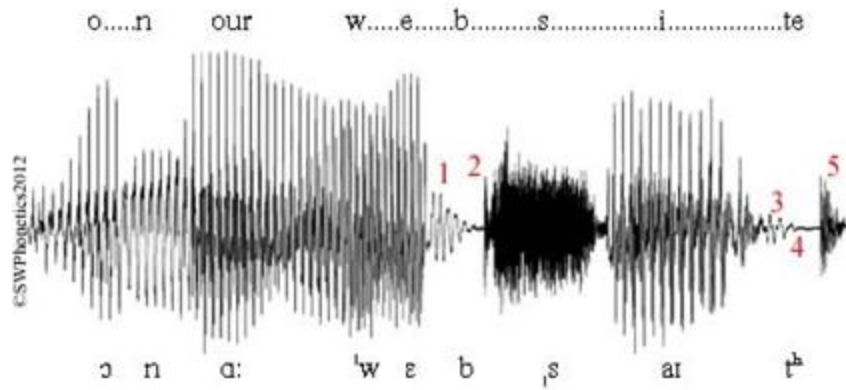


Natural language processing & language modelling

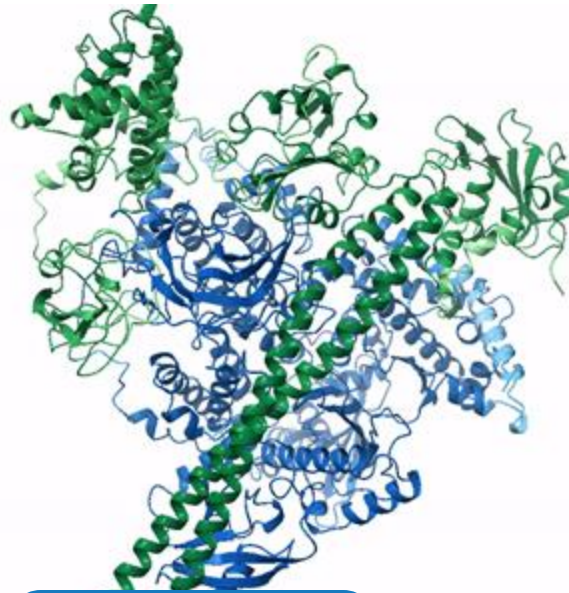
Simone van Bruggen

High Performance Machine Learning

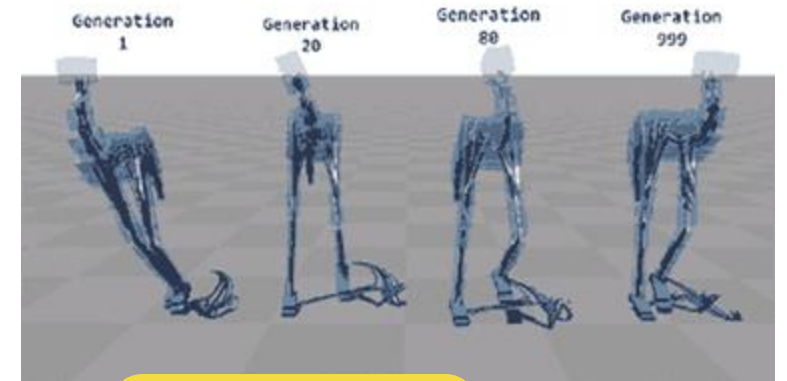
| Sequential data



Speech

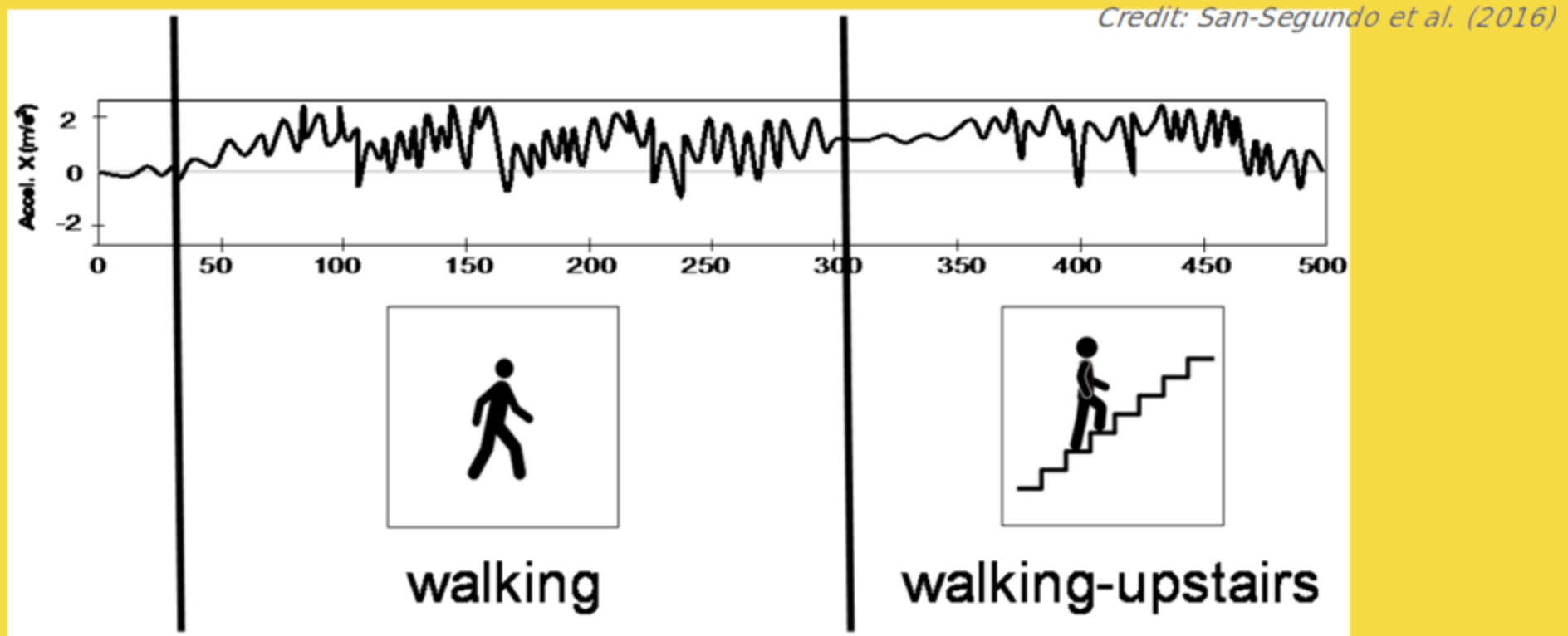


Molecules



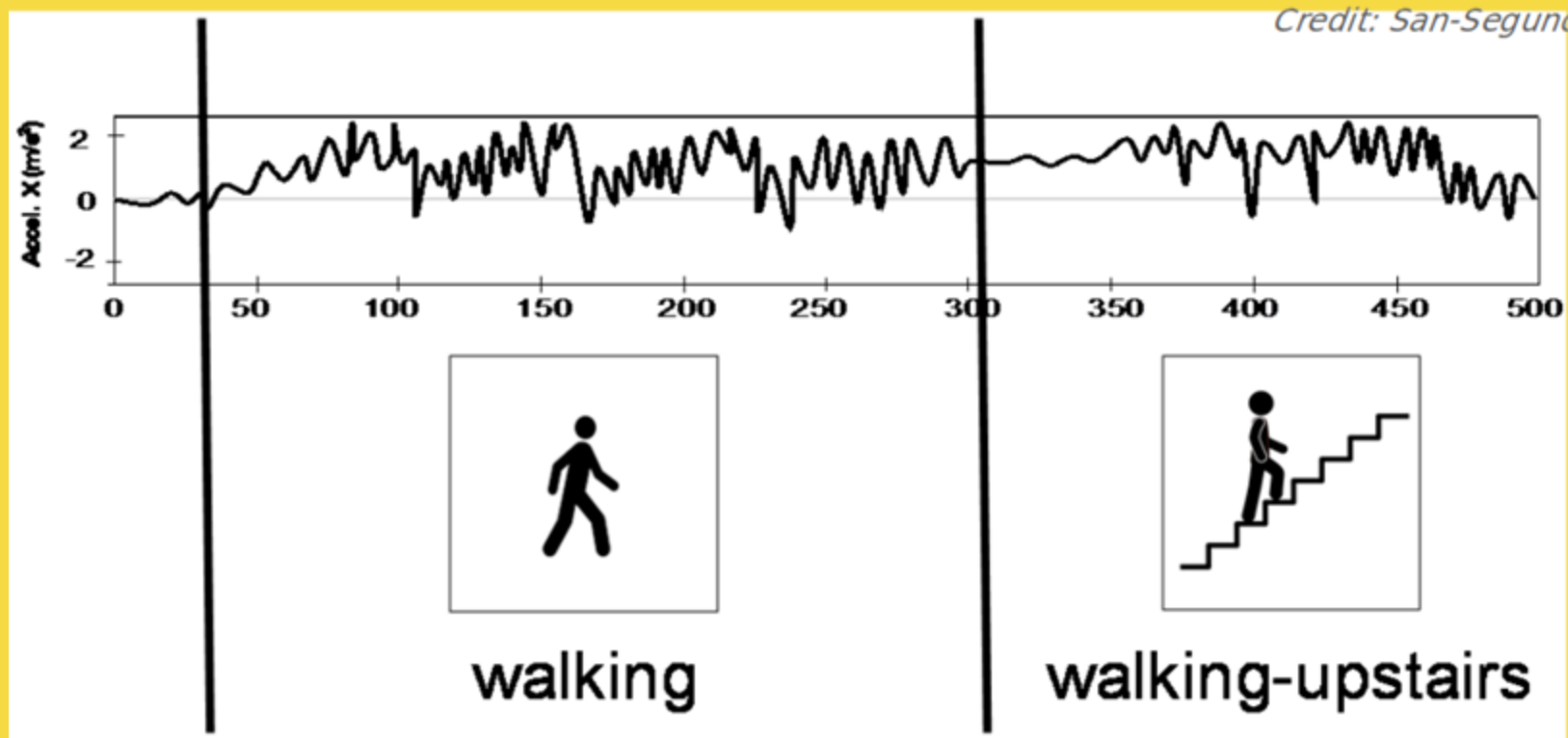
Reinforcement learning

Example: human activity recognition



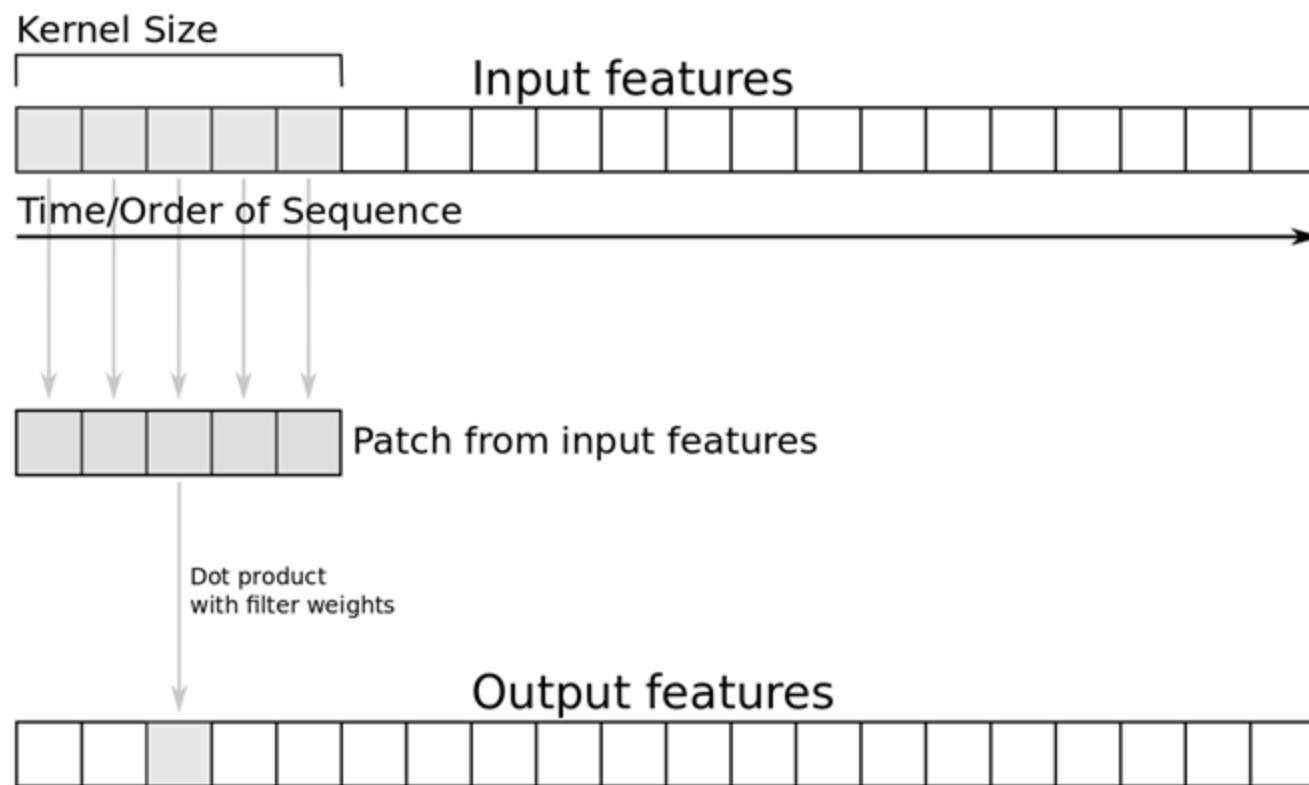
Example: human activity recognition

Can we use a CNN?



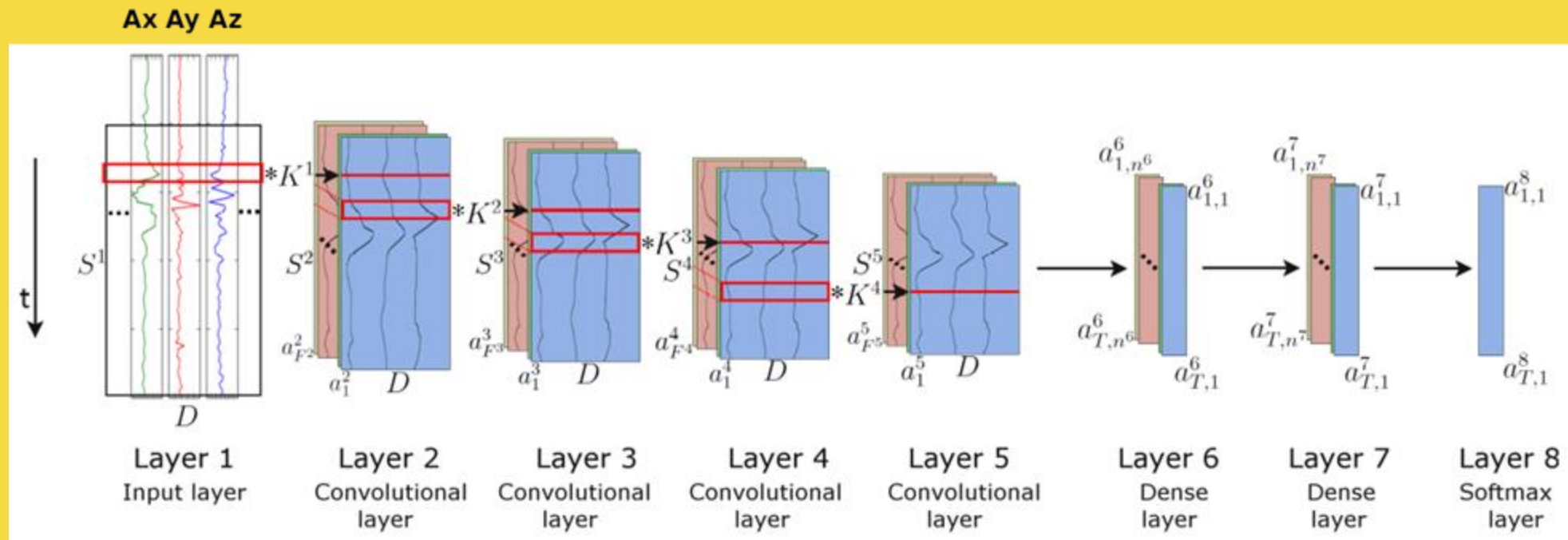
Example: human activity recognition

Can we use a CNN? CNNs work here because they take nearby tokens into account



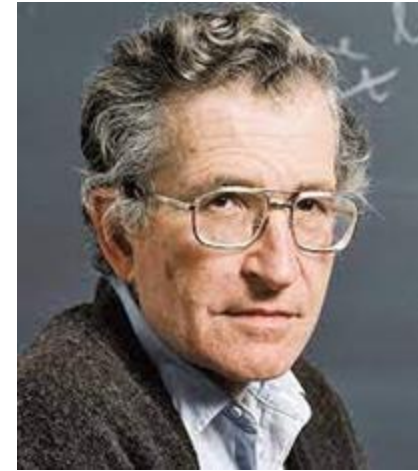
Example: human activity recognition

Can we use a CNN? CNNs work here because they take nearby tokens into account



| How can we model language?

- Is there one (sequential) model to rule them all?
- How does modelling language differ from other sequences?
- Do humans have some specific, innate ability to understand language?



Modern language models refute Chomsky's approach to language

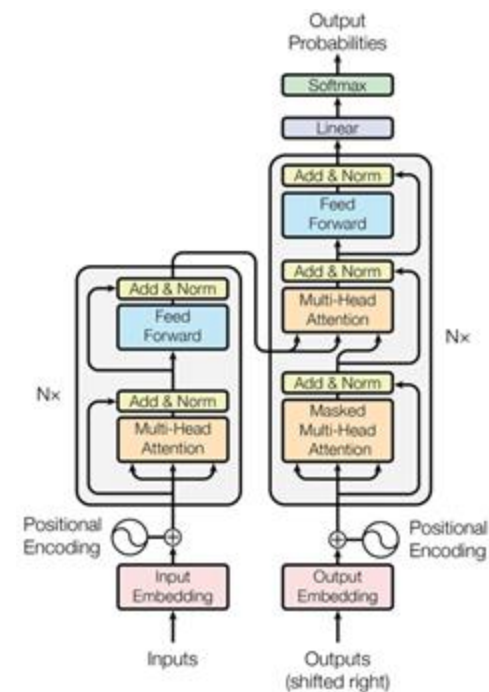
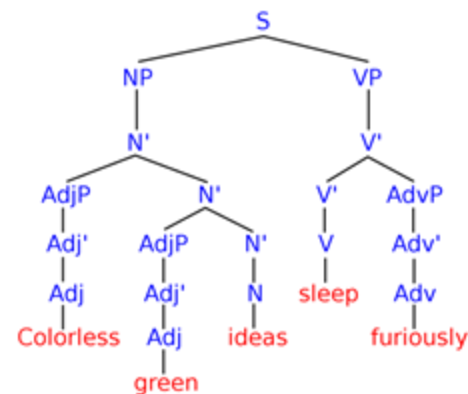
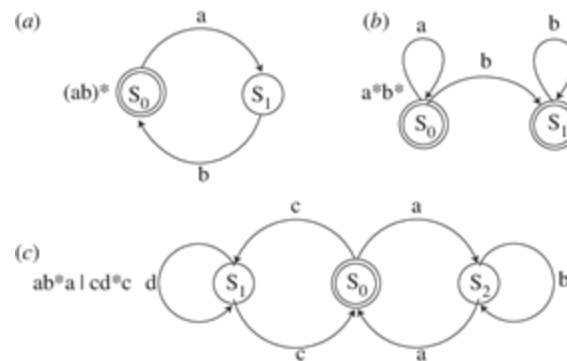
Steven T. Piantadosi^{a,b}

^aUC Berkeley, Psychology ^bHelen Wills Neuroscience Institute

| How can we model language?

Different ways to model language:

- Finite State Automata
- Generative Grammars
- Deep Learning + data
-



| How can we model language?

- Modelling *natural language*
- Language modelling allows us to:
 - Generate new sentences (e.g. chatbot, machine translation)
 - Analyse text (e.g. language identification, classification)
 - Retrieve documents
 - Etc.

| The puzzle pieces

Words

How do we represent the different strings that make up a sentence?

Long-range dependencies

How do we “remember” what was said at the beginning of the sentence?



Data

How do we learn these patterns?

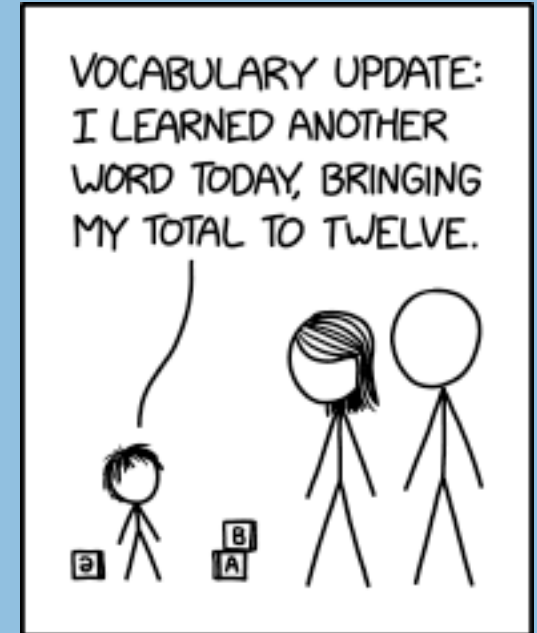
Focus

What part of the sentence is most important?

| 1) How do we represent words?

Language is discrete

We need a vocabulary (set of unique tokens in your language or dataset)



| 1) How do we represent words?

“

*One cat ate a tuna,
one cat ate a
chicken, one weird
cat ate a human.*

| 1) How do we represent words?

“

*One cat ate a tuna,
one cat ate a
chicken, one weird
cat ate a human.*

Vocabulary

one	0
cat	1
ate	2
tuna	3
chicken	4
weird	5
human	6
a	7

1) How do we represent words?

“
One cat ate a tuna,
one cat ate a
chicken, one weird
cat ate a human.

Vocabulary

one
cat
ate
tuna
chicken
weird
human
a

0
1
2
3
4
5
6
7



Embedding

[0.122, 1.54, -0.193]

[0.567, 0.32, 1.243]

1) How do we represent words?

“*One cat ate a tuna,
one cat ate a
chicken, one weird
cat ate a human.*”

Easy to **compose** and
easy to **vectorize**

Vocabulary

one
cat
ate
tuna
chicken
weird
human
a

$\begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{bmatrix}$



$[0.122, 1.54, -0.193]$

$[0.567, 0.32, 1.243]$

| 1) Word embeddings

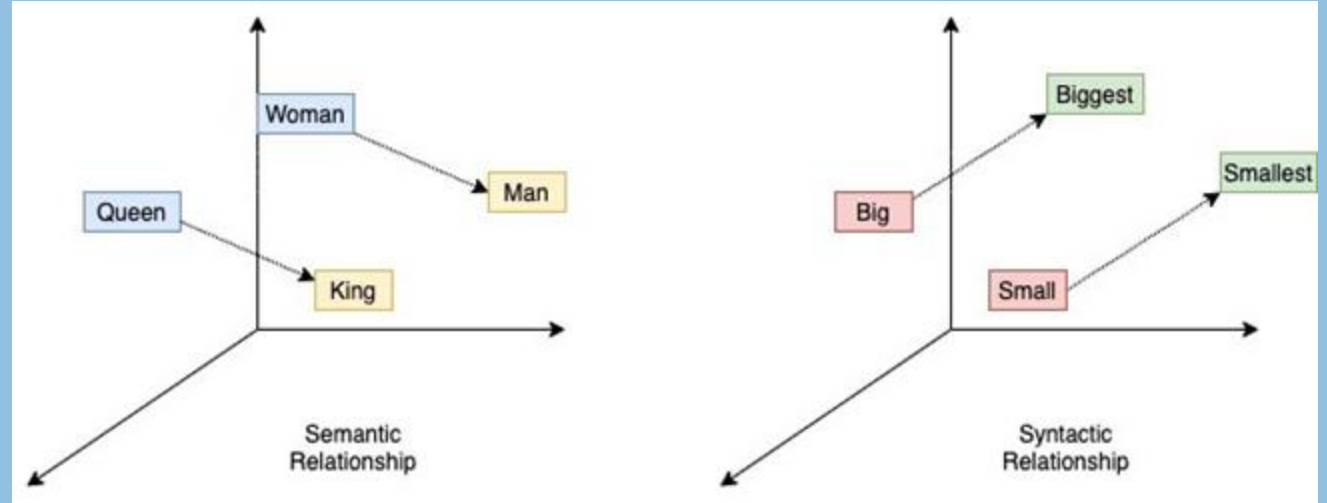
Distributional hypothesis

- The **context** surrounding a given word provides information about its meaning
- Words are similar if they share similar **contexts** (semantic + syntactic)
- Semantic similarity $\approx$ distributional similarity

1) Word embeddings

Distributional hypothesis

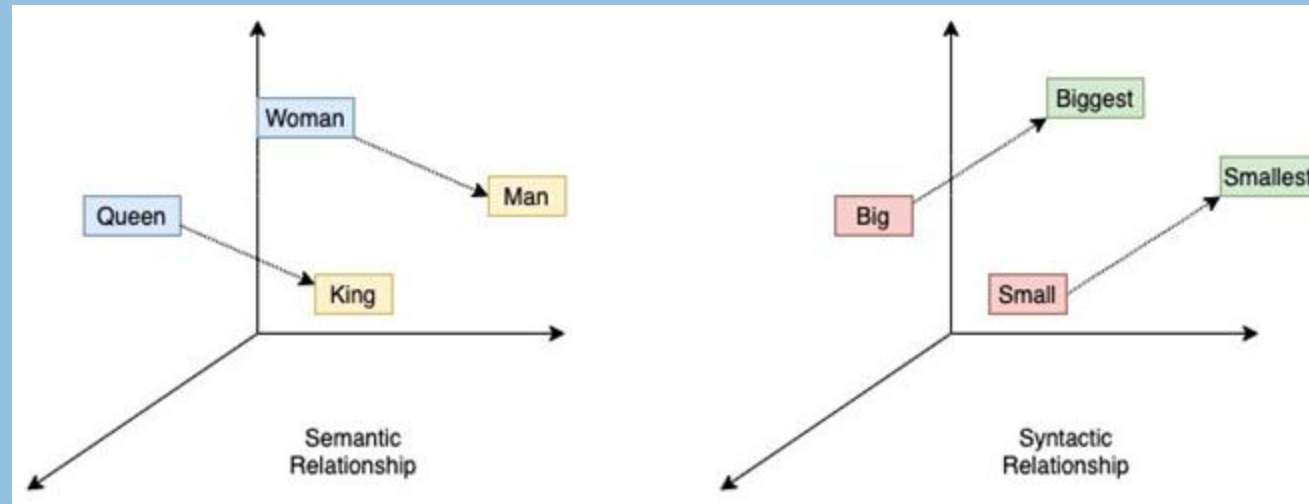
- The **context** surrounding a given word provides information about its meaning
- Words are similar if they share similar **contexts** (semantic + syntactic)
- Semantic similarity $\approx$ distributional similarity



1) Word embeddings

Distributional hypothesis

- The **context** surrounding a given word provides information about its meaning
- Words are similar if they share similar **contexts** (semantic + syntactic)
- Semantic similarity $\approx$ distributional similarity



Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Miami - Florida	Baltimore: Maryland	Dallas: Texas	Kona: Hawaii
Einstein - scientist	Messi: midfielder	Mozart: violinist	Picasso: painter
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev	Obama: Barack
Microsoft - Windows	Google: Android	IBM: Linux	Apple: iPhone
Microsoft - Ballmer	Google: Yahoo	IBM: McNealy	Apple: Jobs
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

| 2) How do we learn from data?

“The monkey at that banana because it was ...”

| 2) How do we learn from data?

“The monkey at that banana because it was ...”

$$P(w_1, w_2, \dots w_n) = P(w_1)P(w_2 | w_1)P(w_3 | w_1, w_2) \dots P(w_n | w_1, \dots w_{n-1})$$

Language modelling task

Unsupervised learning: predict the next word in a sentence by **masking**

| 2) How do we learn from data?

“The monkey at that banana because it was ...”

$$P(w_1, w_2, \dots w_n) = P(w_1)P(w_2 | w_1)P(w_3 | w_1, w_2) \dots P(w_n | w_1, \dots w_{n-1})$$

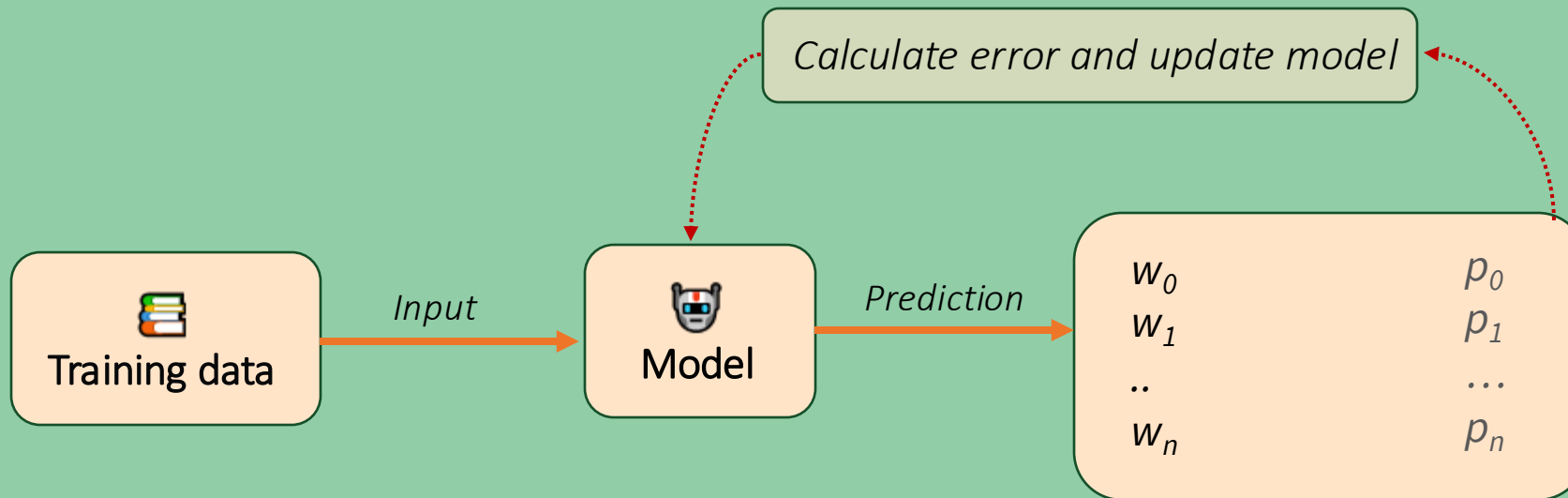
Language modelling task

Unsupervised learning: predict the next word in a sentence by **masking**

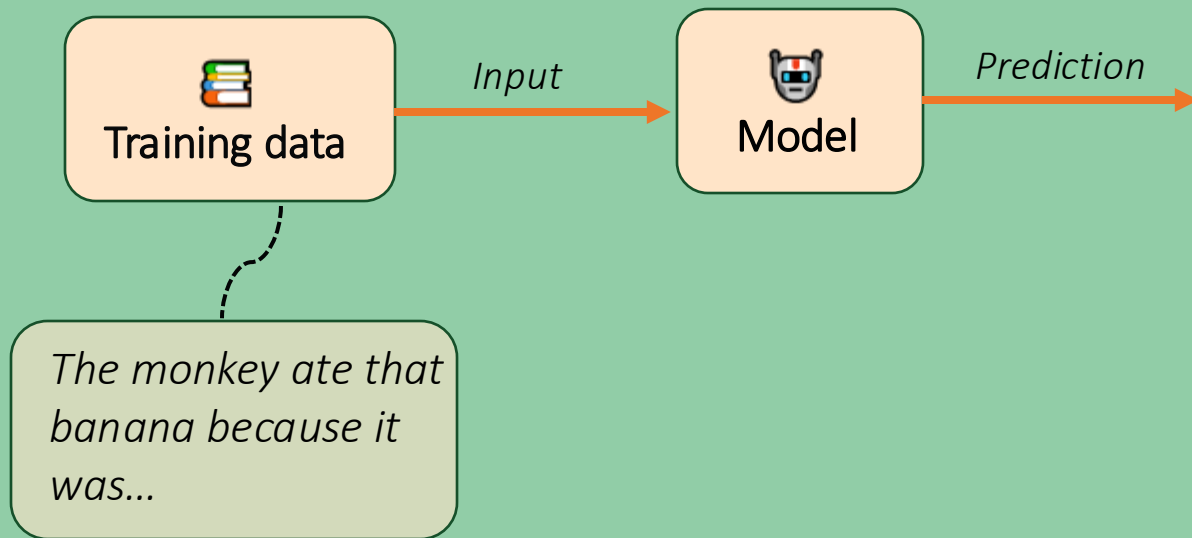
Language model objective

-  Model the **probability** of the next symbol given the previous context by creating a probability distribution over language
-  Language models like ChatGPT model language *left to right*: one word at a time

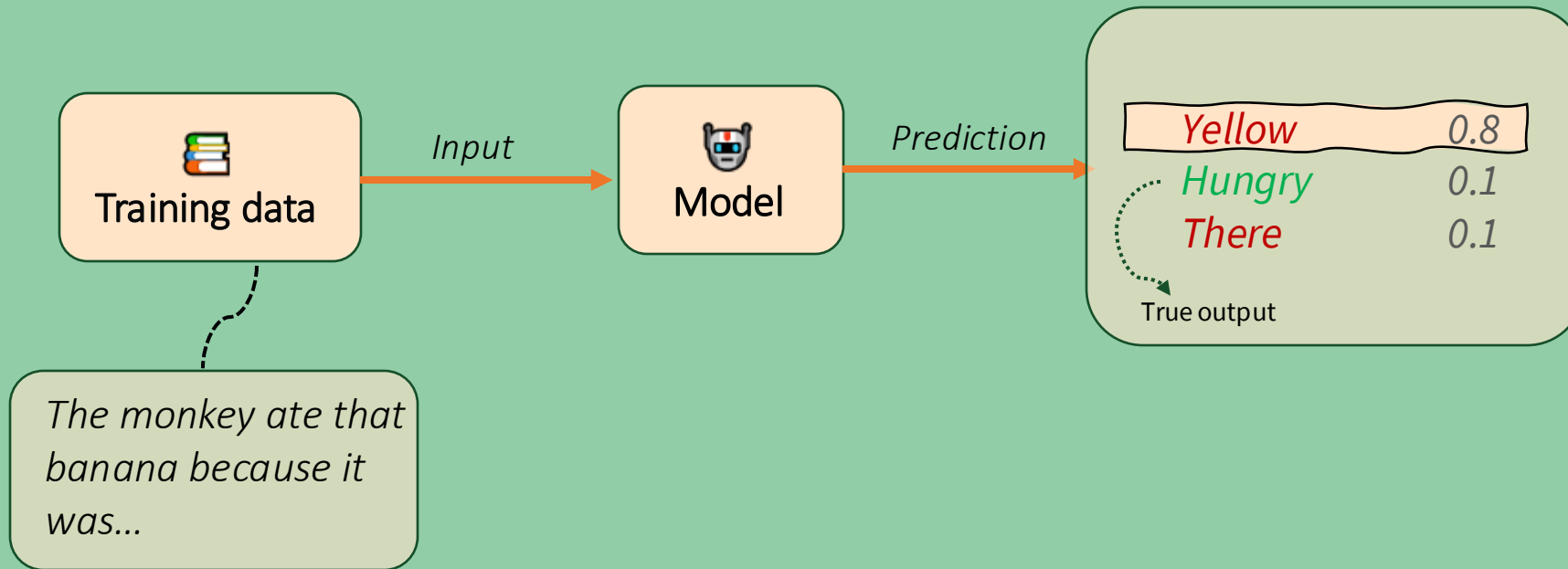
2) How do we learn from data?



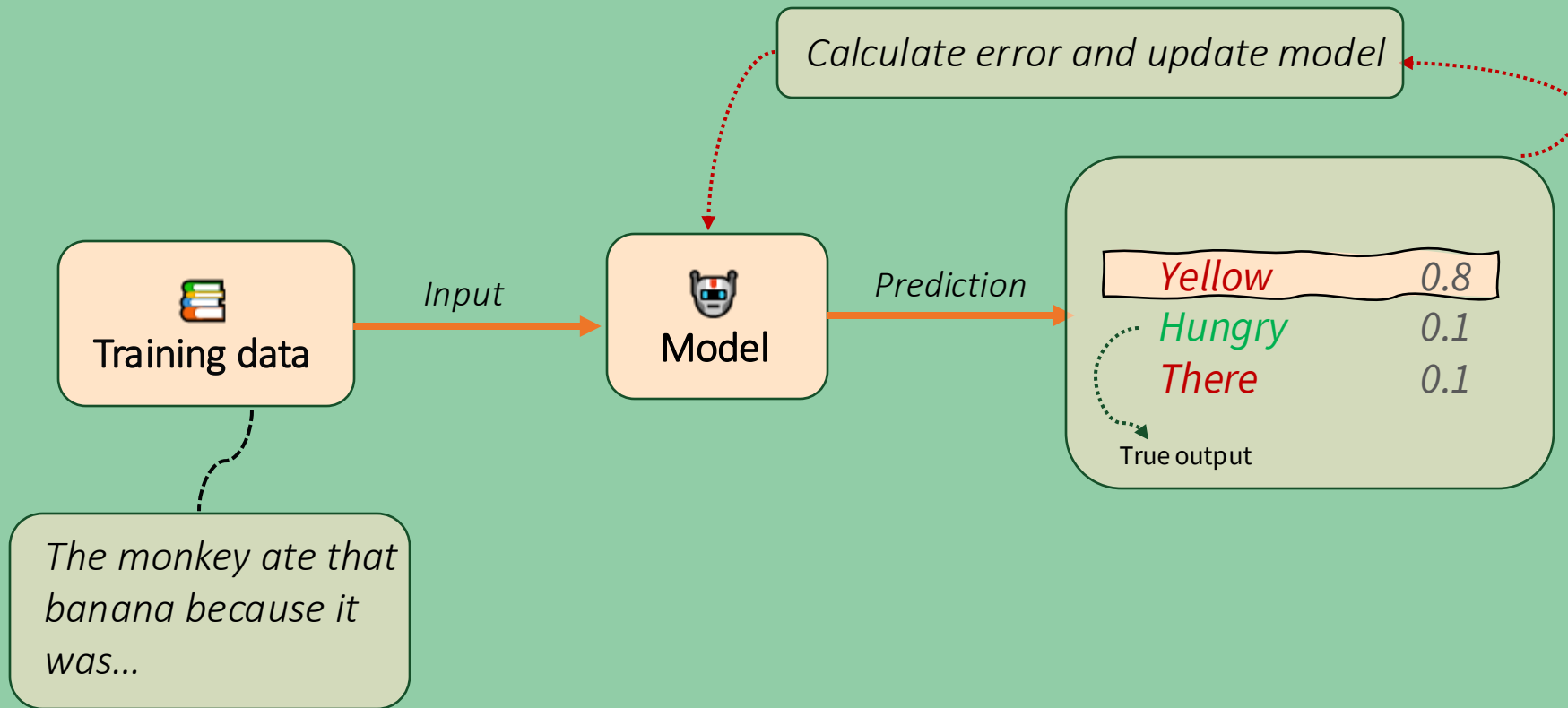
| 2) How do we learn from data?



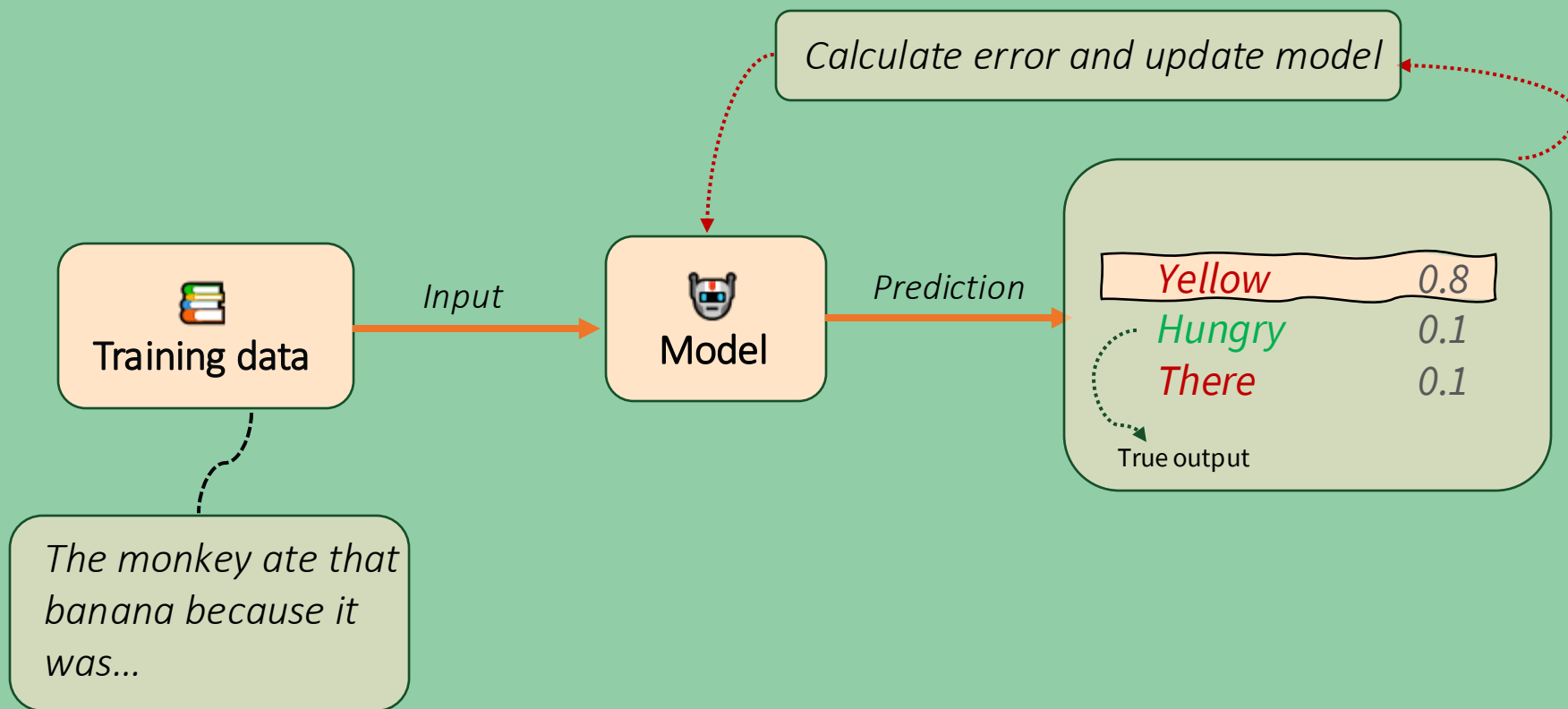
| 2) How do we learn from data?



| 2) How do we learn from data?



| 2) How do we learn from data?

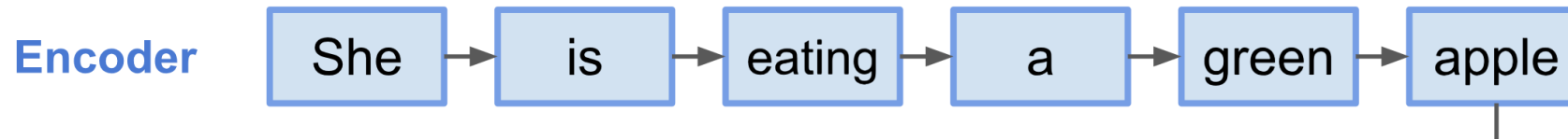


Can we use CNNs for this?

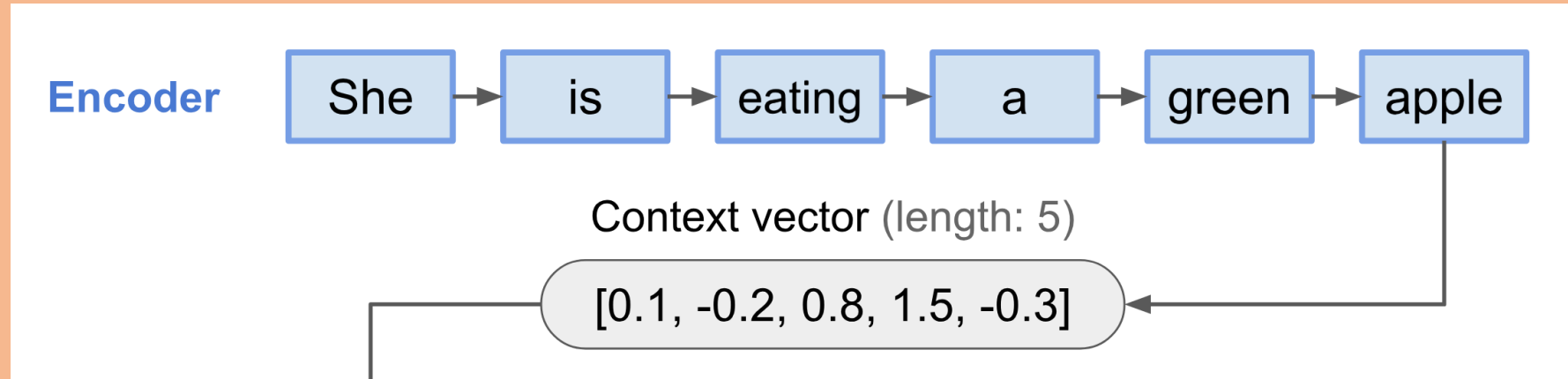
CNN context size is small and does **not** *remember* past context

Language is very noisy and heavily (long-distance) context dependent

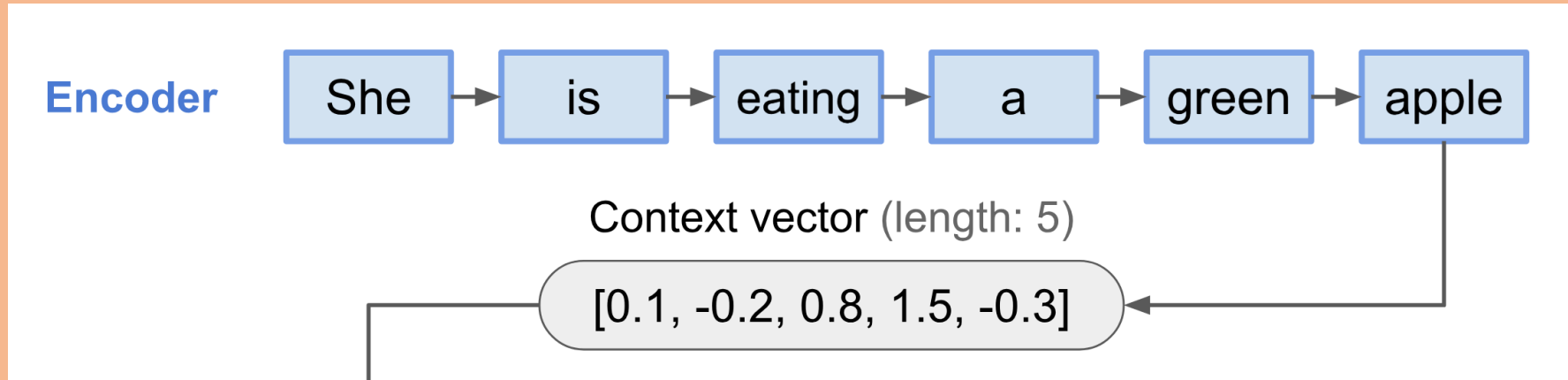
3) How do we model long-range dependencies?



3) How do we model long-range dependencies?



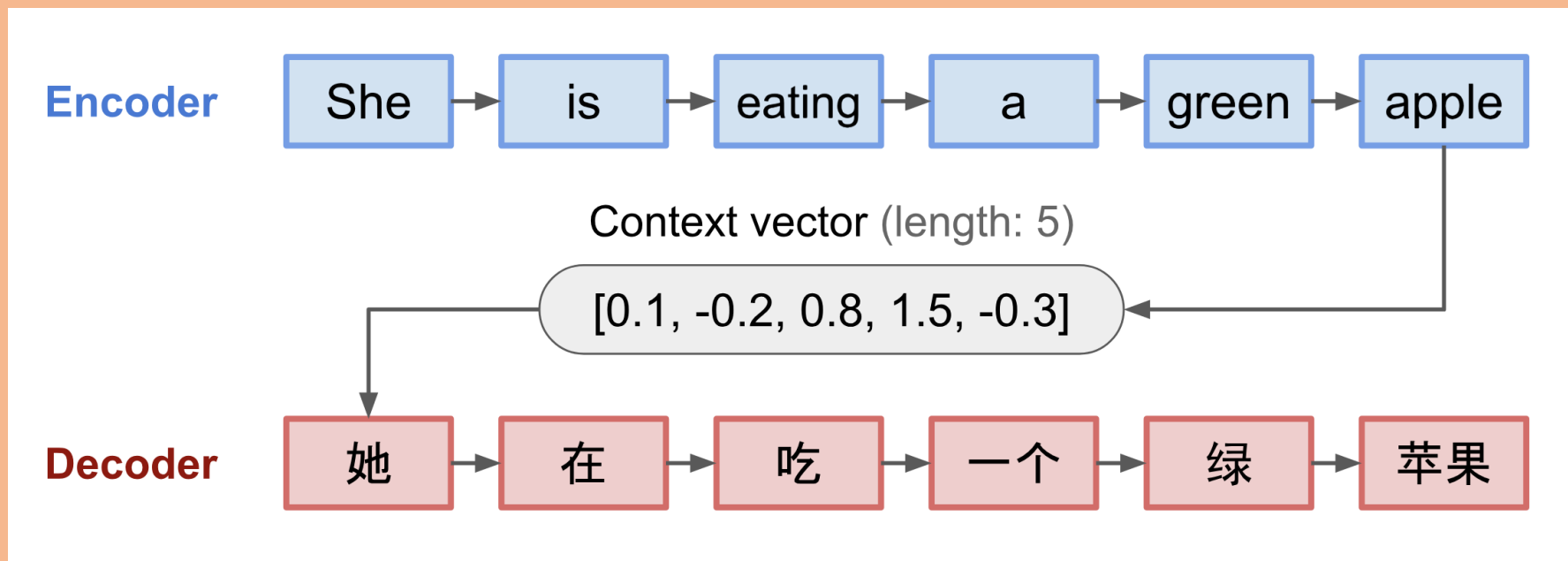
3) How do we model long-range dependencies?



Use some type of recurrent network: RNN, LSTM

- Classify using the *context vector*: “summary” of the input string
 - Usually last hidden state or average of all states

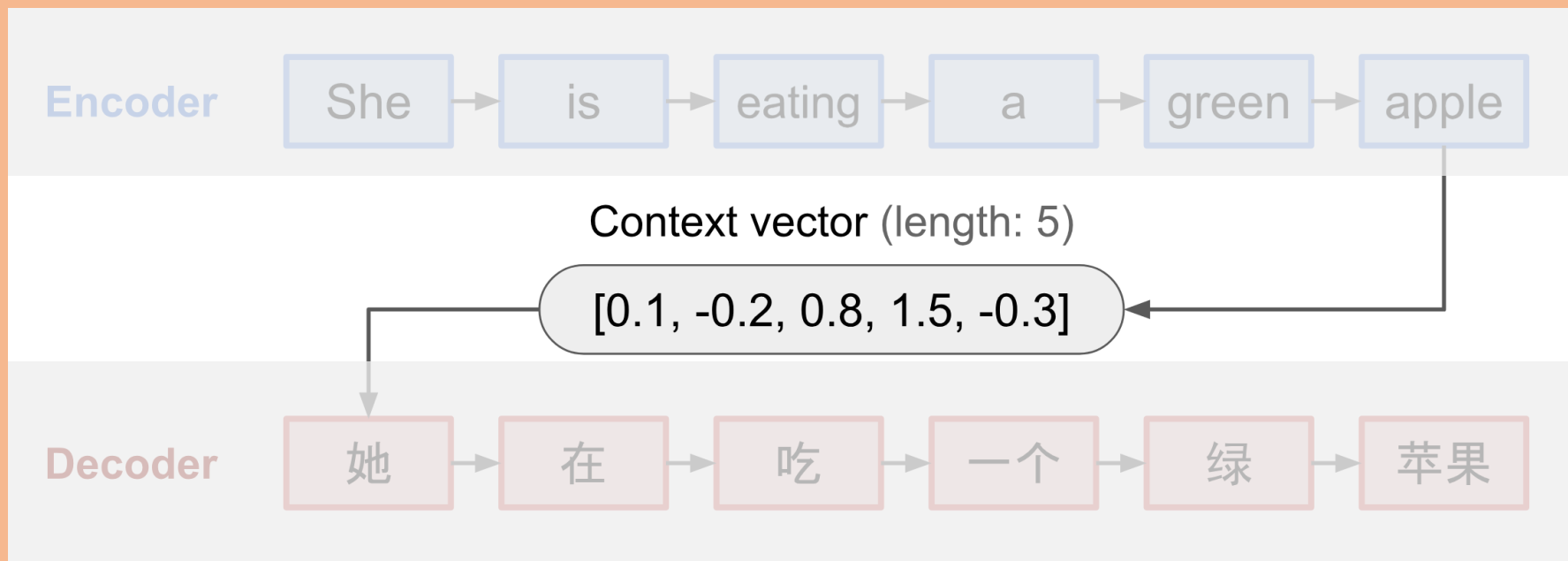
3) How do we model long-range dependencies?



Use some type of recurrent network: RNN, LSTM

- Classify using the *context vector*: "summary" of the input string
 - Usually last hidden state or average of all states
- Model sequence-to-sequence tasks

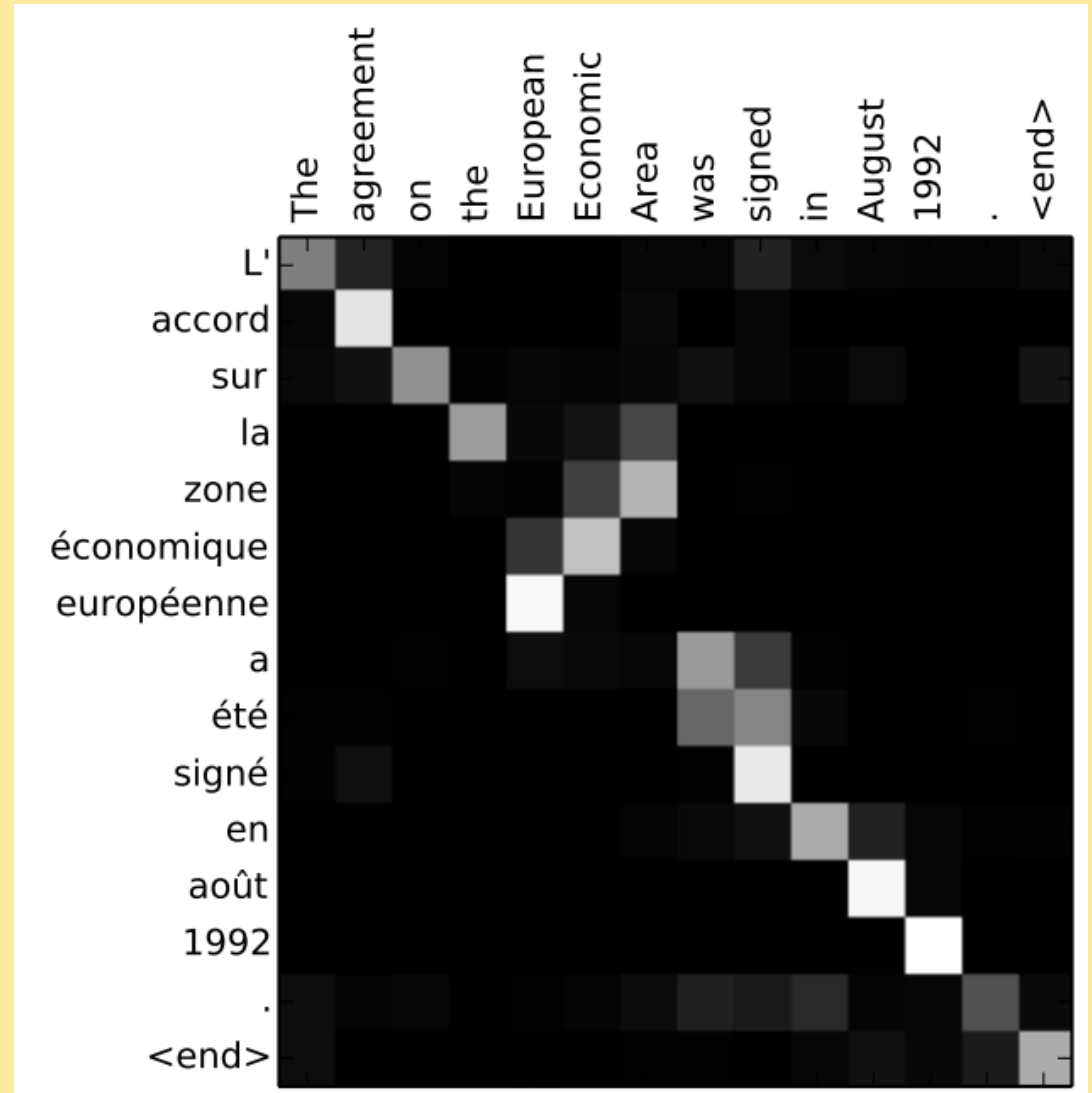
3) How do we model long-range dependencies?



Use some type of recurrent network: RNN, LSTM

- Classify using the *context vector*: "summary" of the input string
 - Usually last hidden state or average of all states
- Model sequence-to-sequence tasks
- Context length is fixed - what would happen if we have the following input:
 - *"Because she was feeling a bit under the weather and she did not have any oranges in the fridge, she is eating a green apple"*

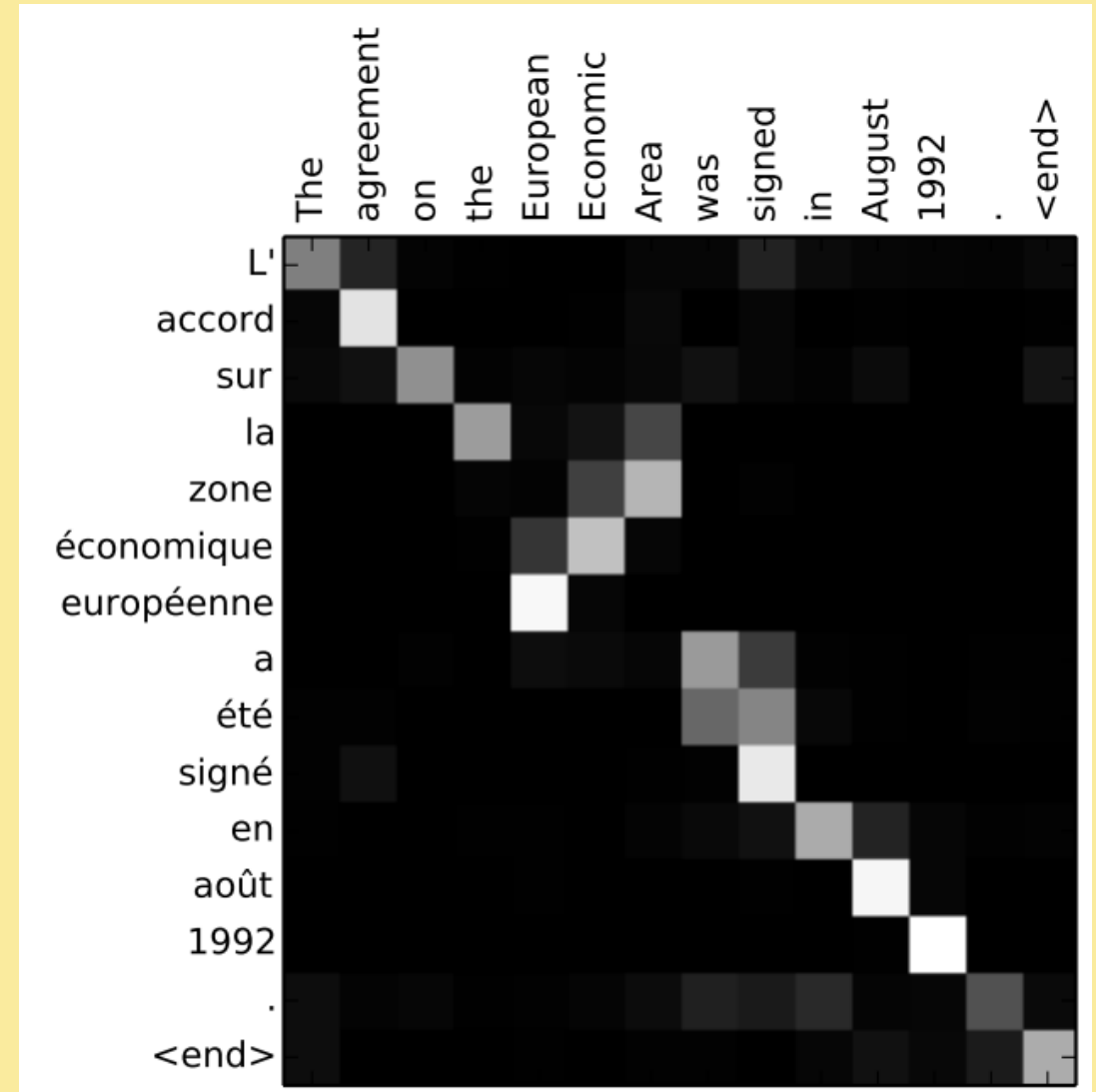
| 4) How do we focus?



| 4) How do we focus?

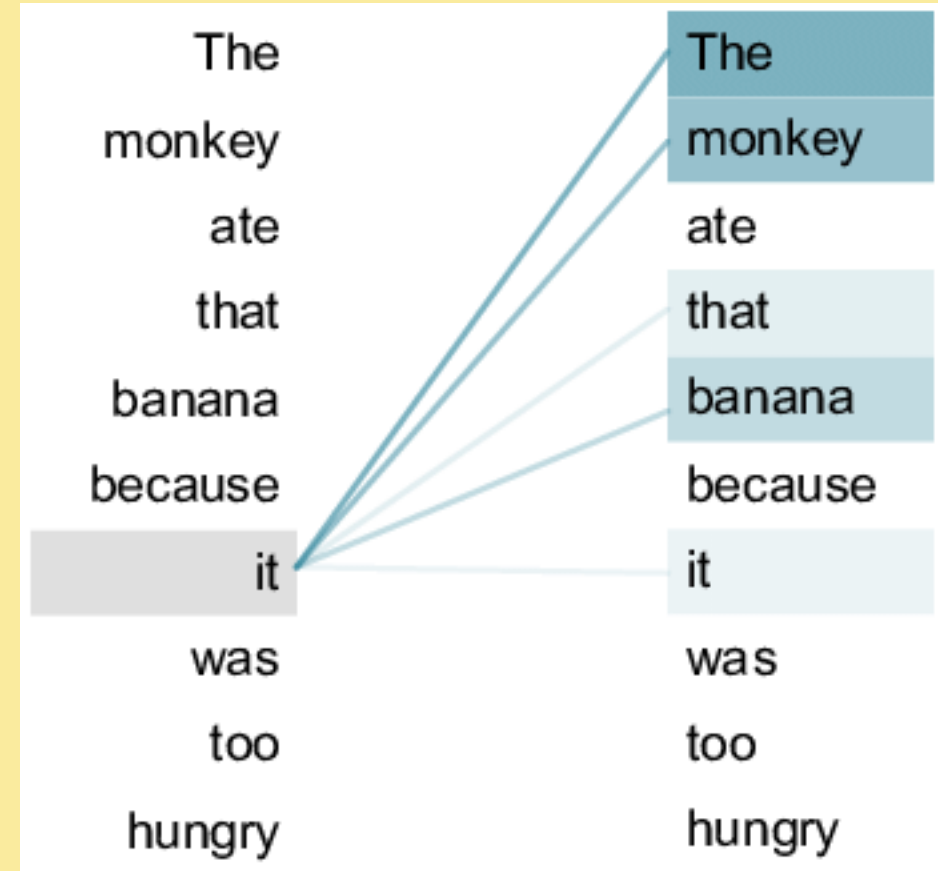
Attention: fixing bottleneck by learning which parts of the input to **attend** to

Pass **all** hidden states to the decoder and use a jointly trained feedforward network to score the importance of each hidden state at each time step



| 4) How do we focus?

Can we also learn which parts to attend to **within** a single sentence?



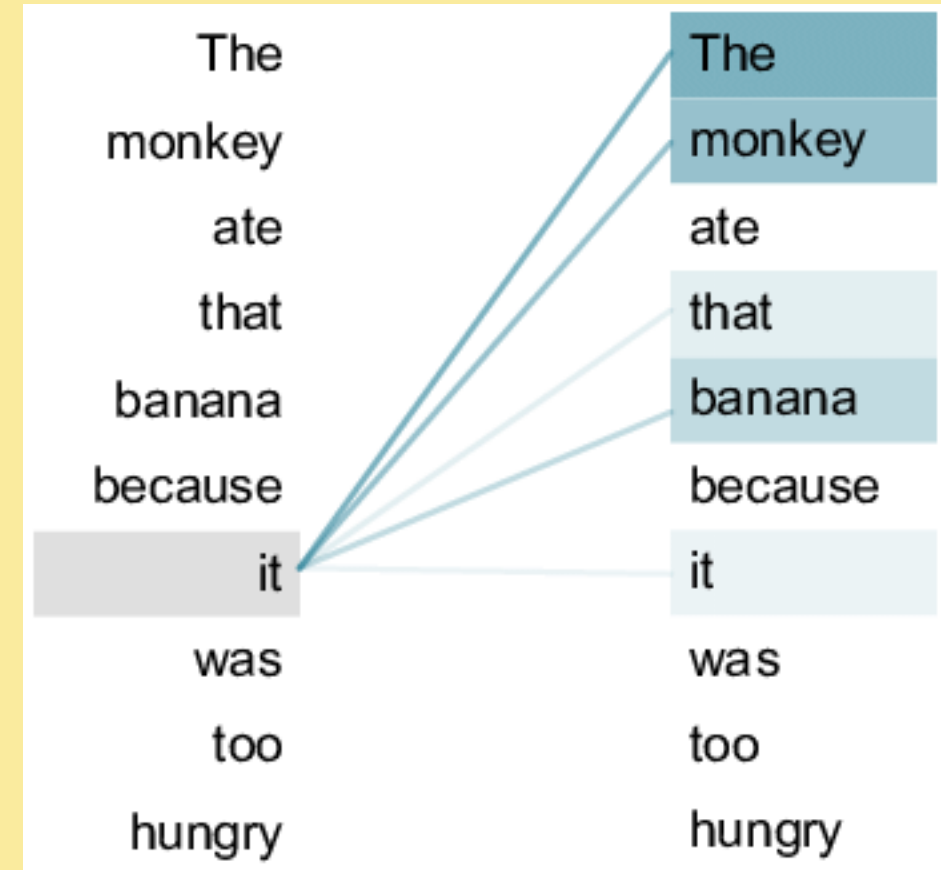
| 4) How do we focus?

Can we also learn which parts to attend to **within** a single sentence?

Yes: self-attention!

What is the **importance** of every element in the input sequence with respect to **itself**?

Every word directly connected to every other word



| Transformers: combining the pieces

No recurrence needed: more parallelization!

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

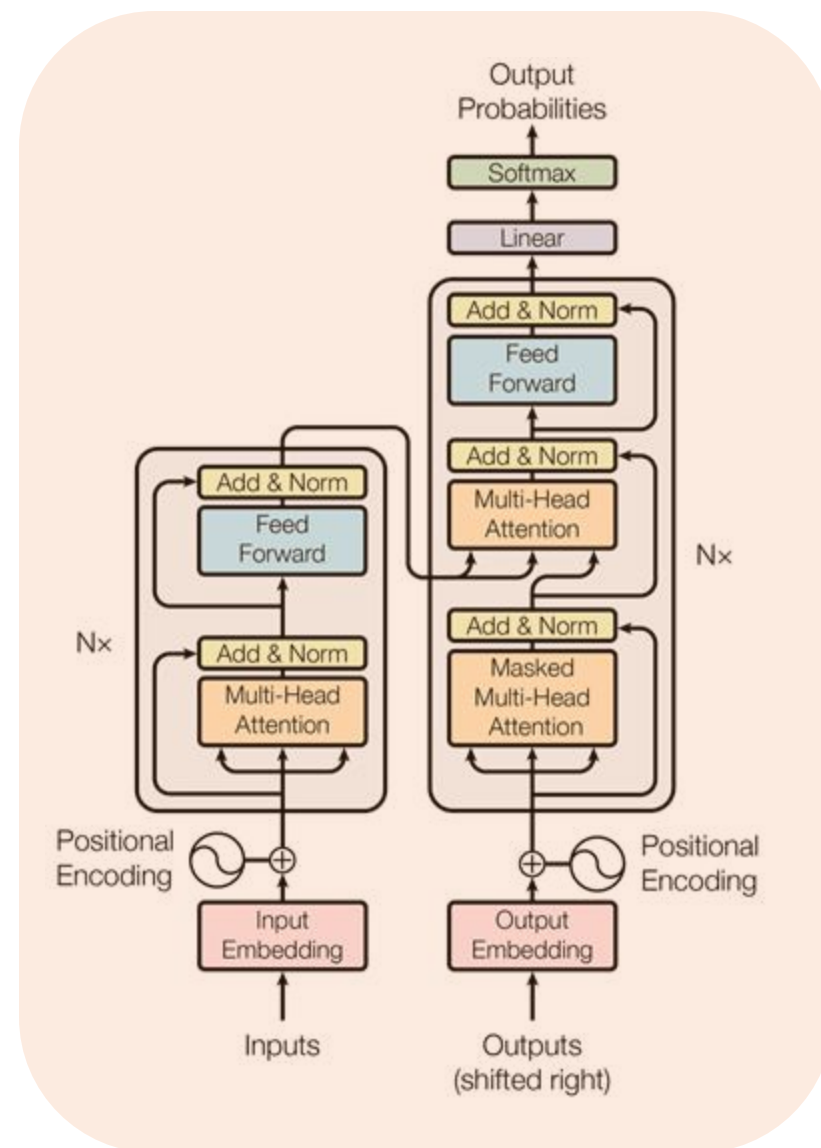
Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

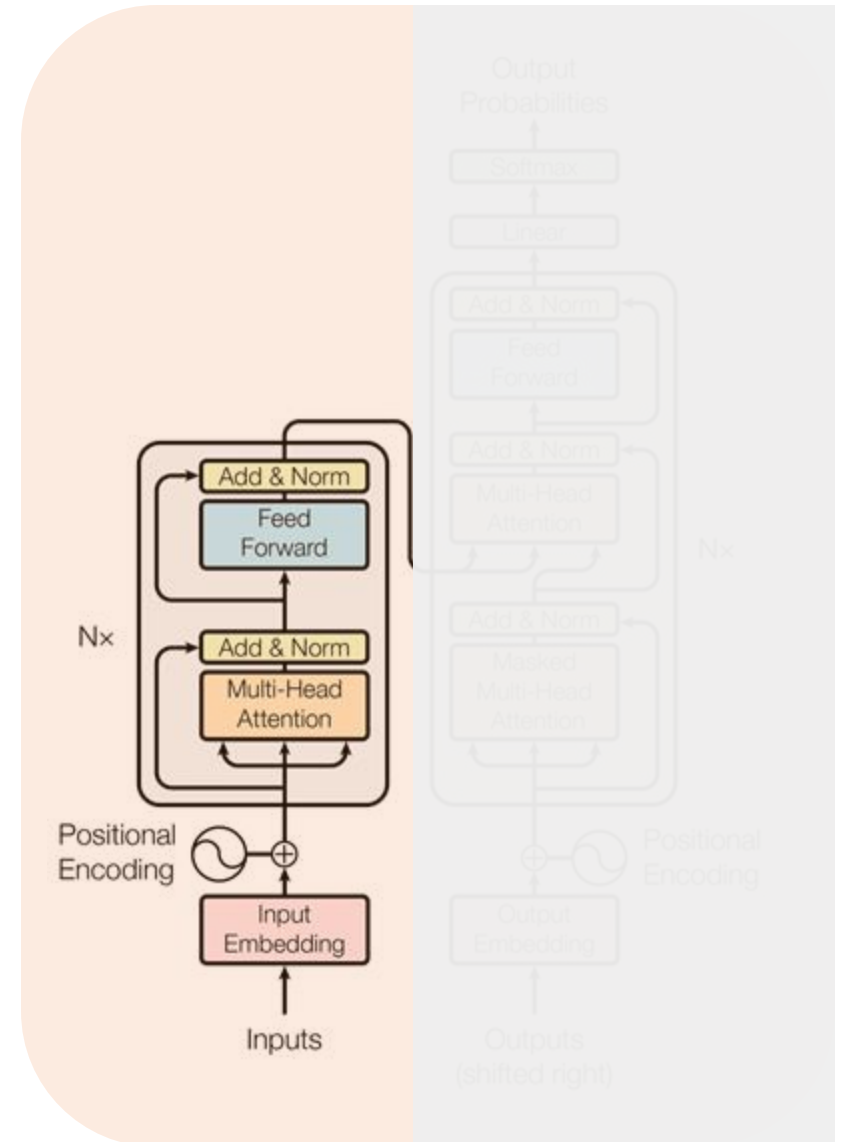
Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com



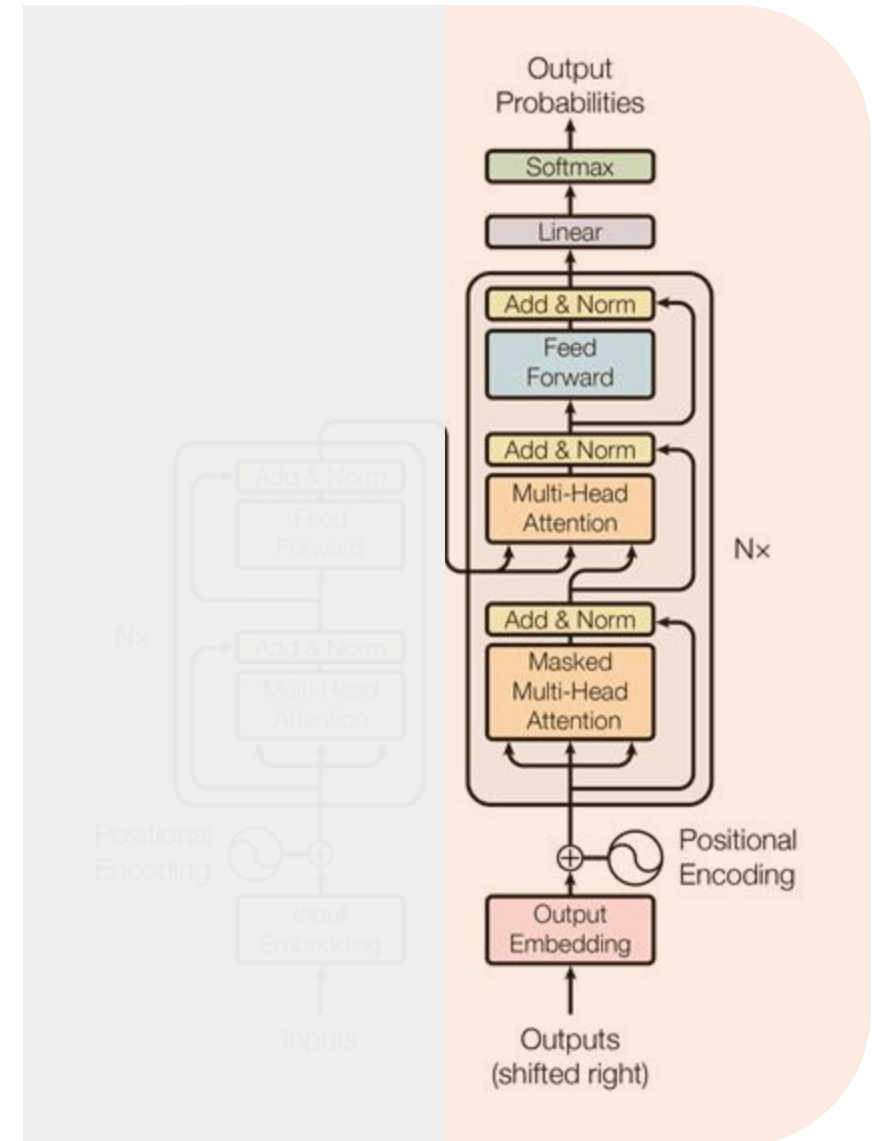
| Transformers: encoder

Understanding the input

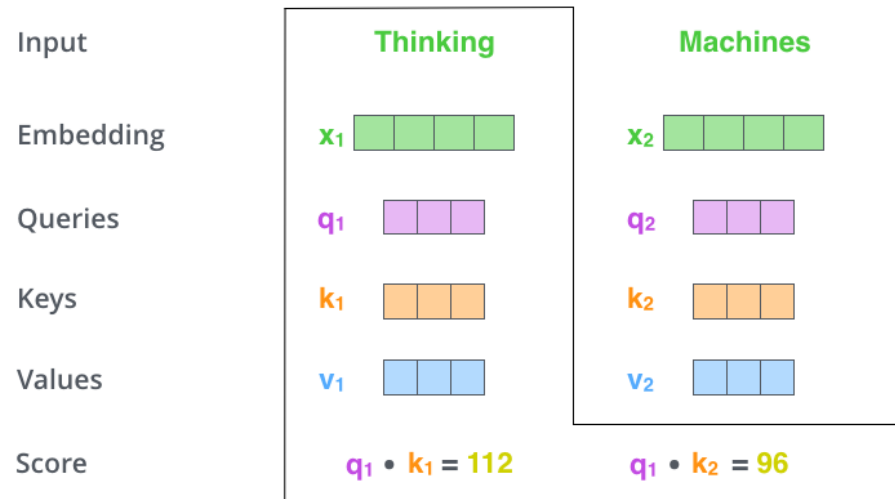


| Transformers: decoder

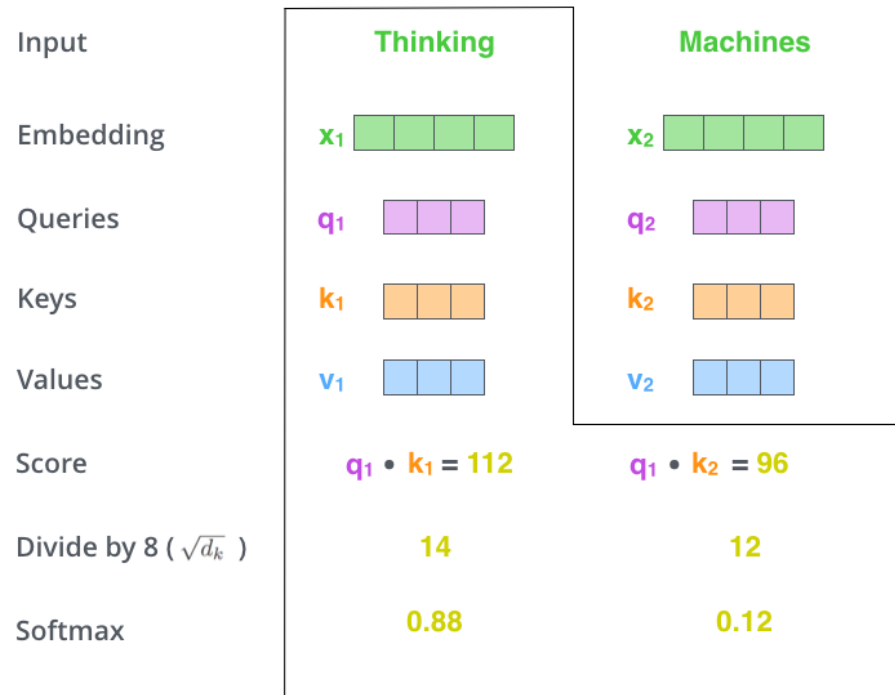
Generating the output



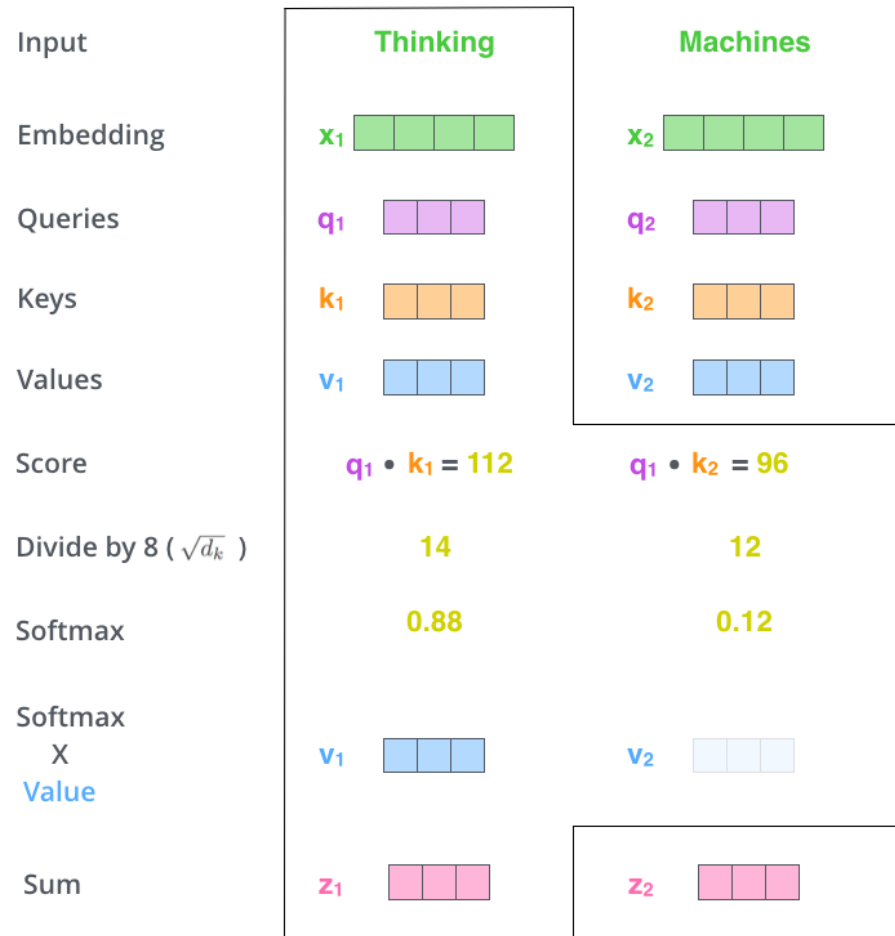
| Transformers: self-attention



| Transformers: self-attention

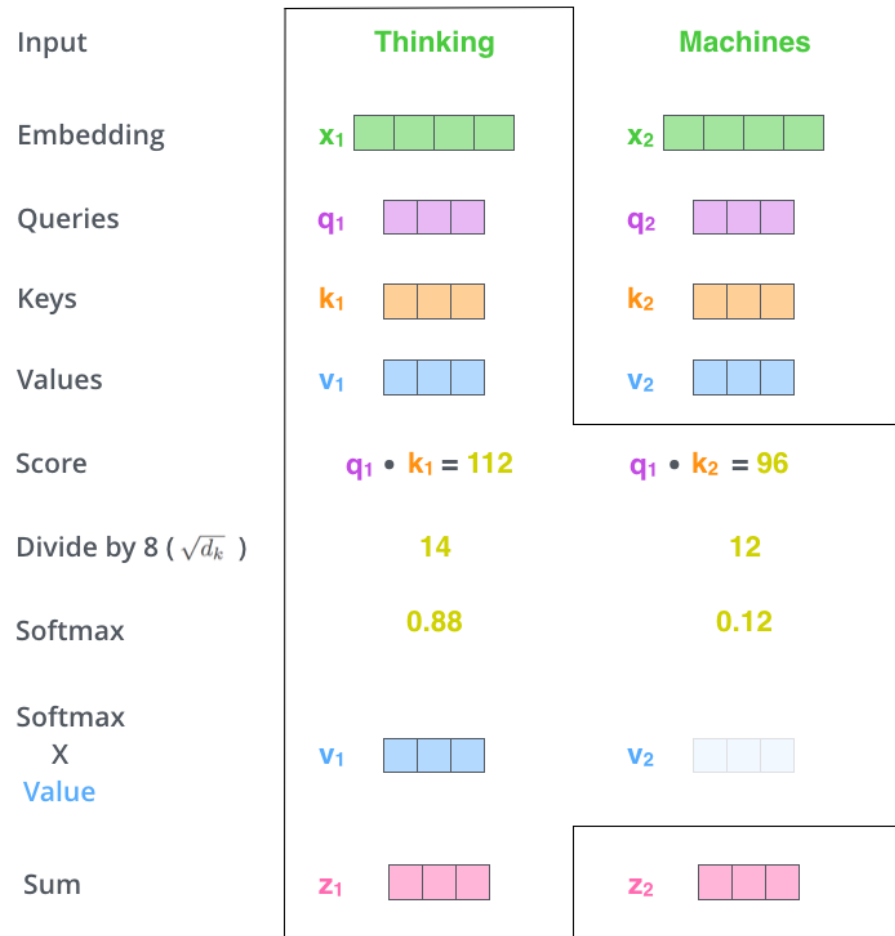


Transformers: self-attention



Transformers: self-attention

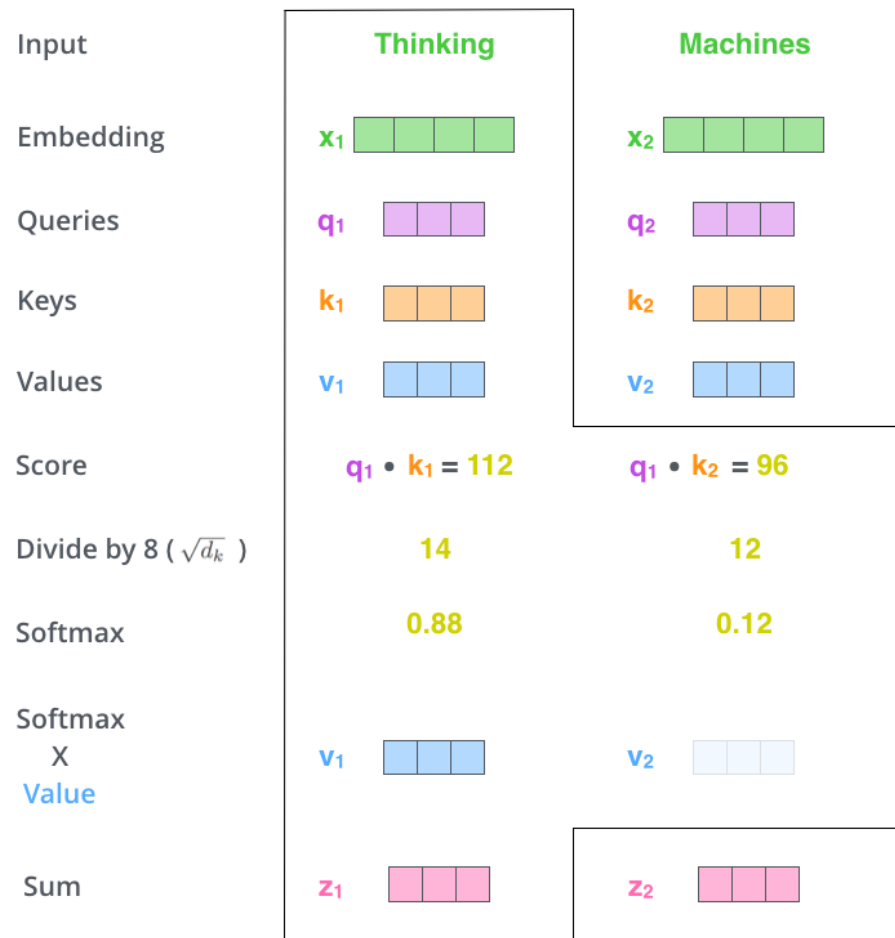
Can be written as matrix form $\rightarrow$ faster!



$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

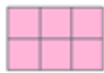
= Z

Transformers: self-attention



Can be written as matrix form $\rightarrow$ faster!

$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$

= Z 

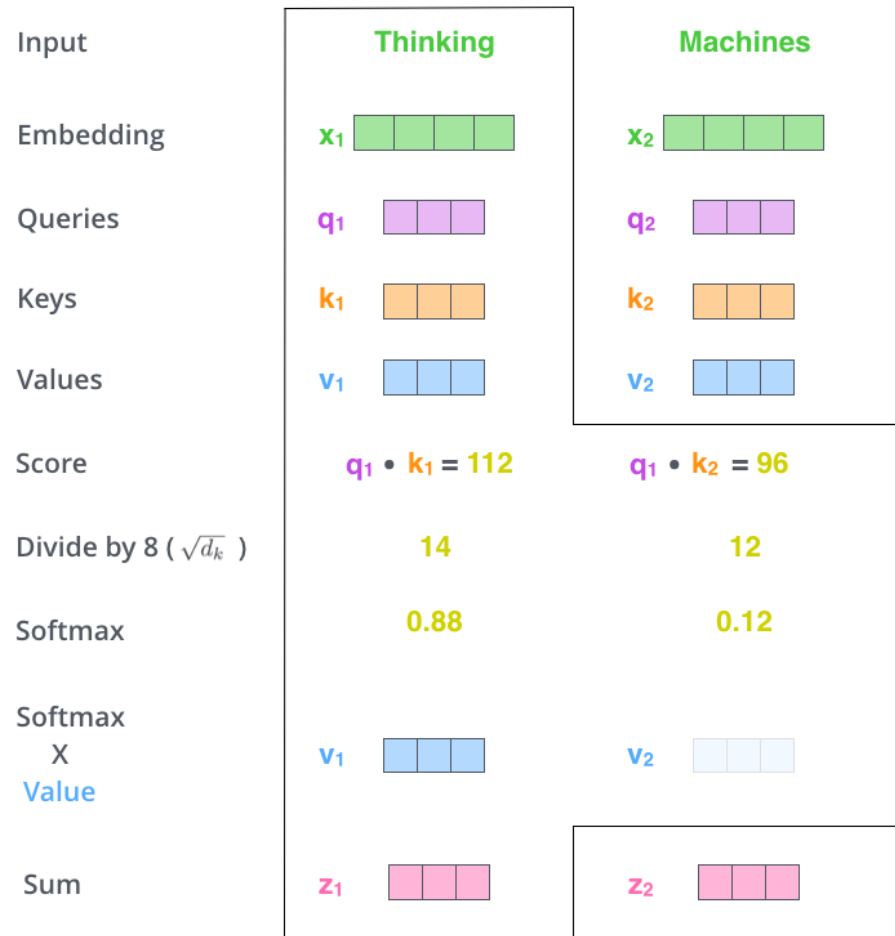
$$\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix}$$

$$\frac{e^{x_i}}{\sum_{j=0}^K e^{x_j}}$$

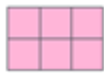
SURF

Transformers: self-attention

Can be written as matrix form → faster!



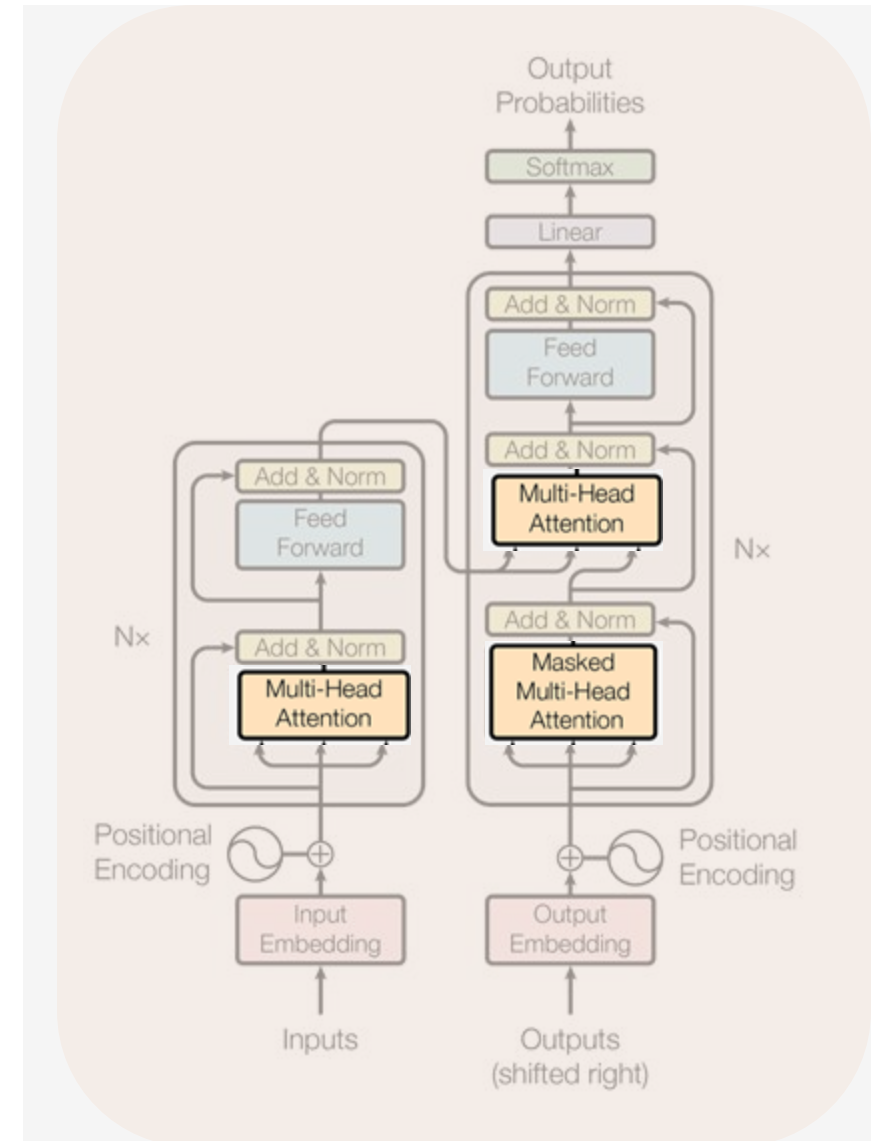
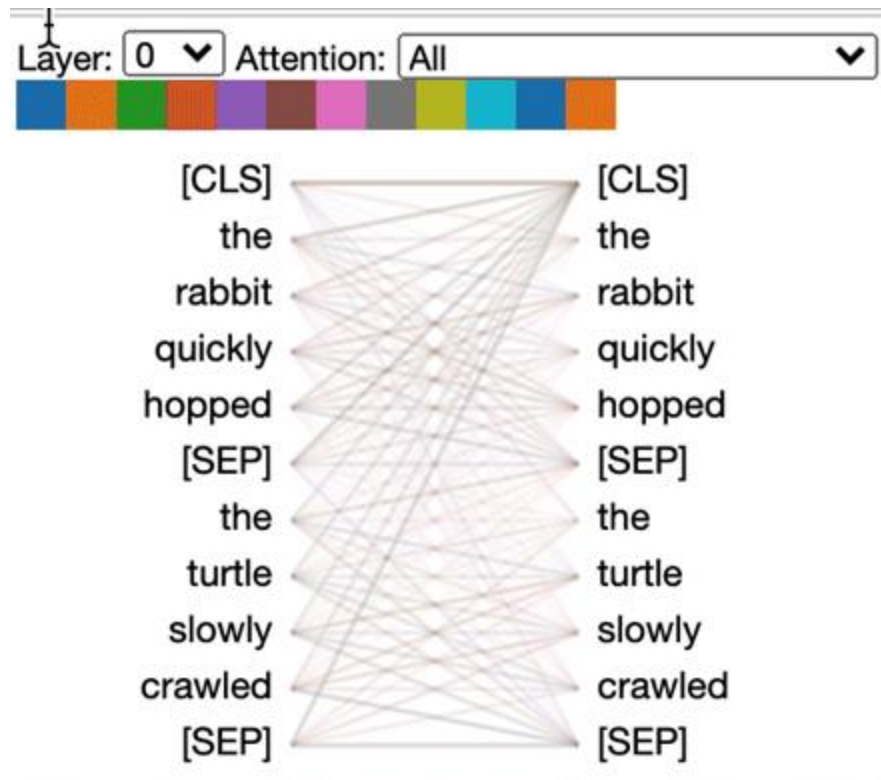
$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$

= Z 

$$\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix} \frac{e^{x_i}}{\sum_{j=0}^K e^{x_j}} \begin{bmatrix} 0.02 \\ 0.90 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$$

SURF

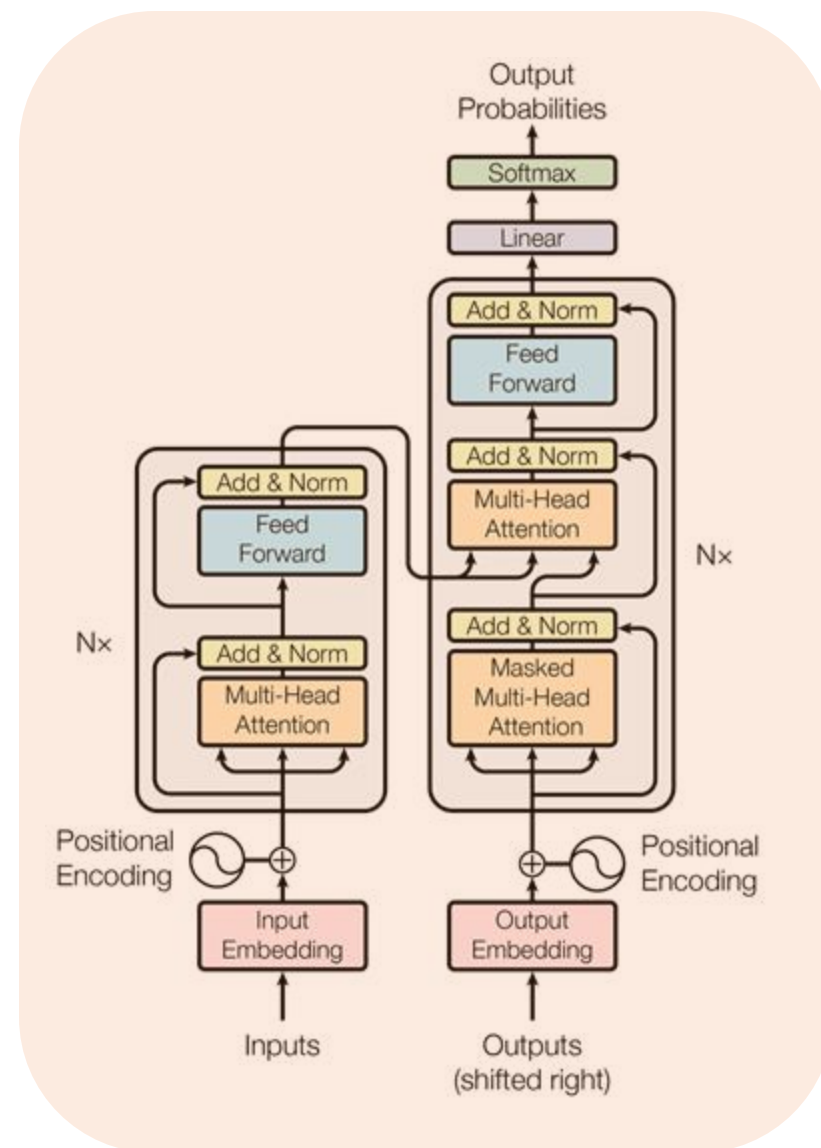
Transformers: multi-head attention



Transformers: positional encodings

Sentences are no longer read sequentially: we need some type of ordering

“The girl chased the dog” vs “The dog chased the girl”



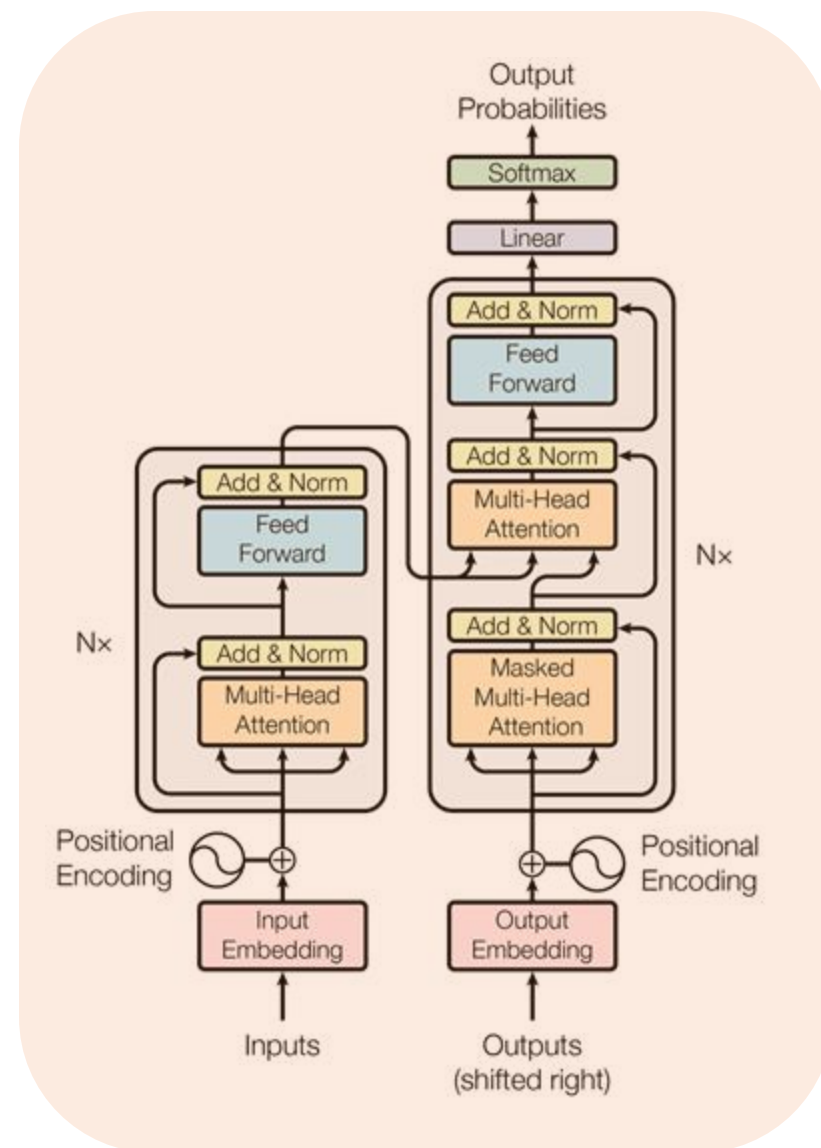
Transformers: positional encodings

- Sentences are no longer read sequentially: we need some type of ordering

“The girl chased the dog” != “The dog chased the girl”

- Positional encoding needs to

Allow for longer sequences during runtime than seen in training
Encode the **relative** position rather than absolute position



Transformers: positional encodings

- Sentences are no longer read sequentially: we need some type of ordering

“The girl chased the dog” != “The dog chased the girl”

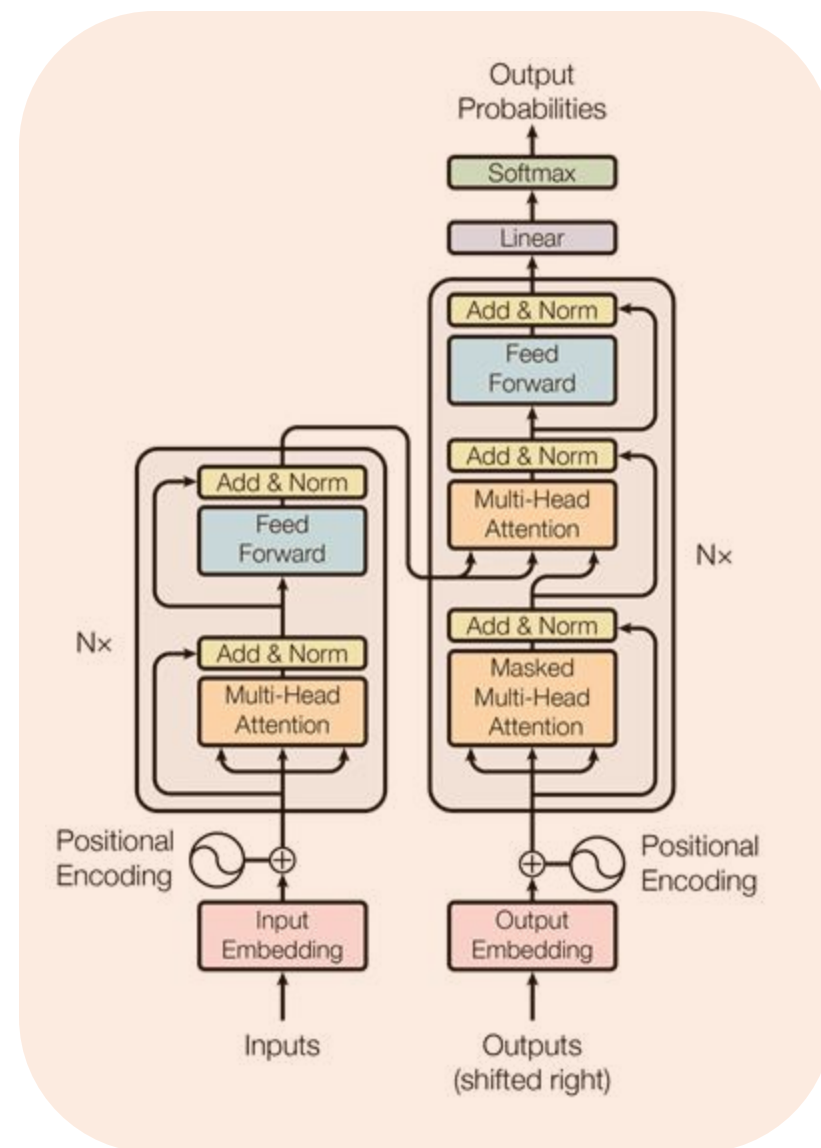
- Positional encoding needs to

Allow for longer sequences during runtime than seen in training
Encode the **relative** position rather than absolute position

Original paper: cosine and sine functions

Modern LLMs: RoPE

<https://huggingface.co/blog/designing-positional-encoding>



| Transformers: the missing pieces

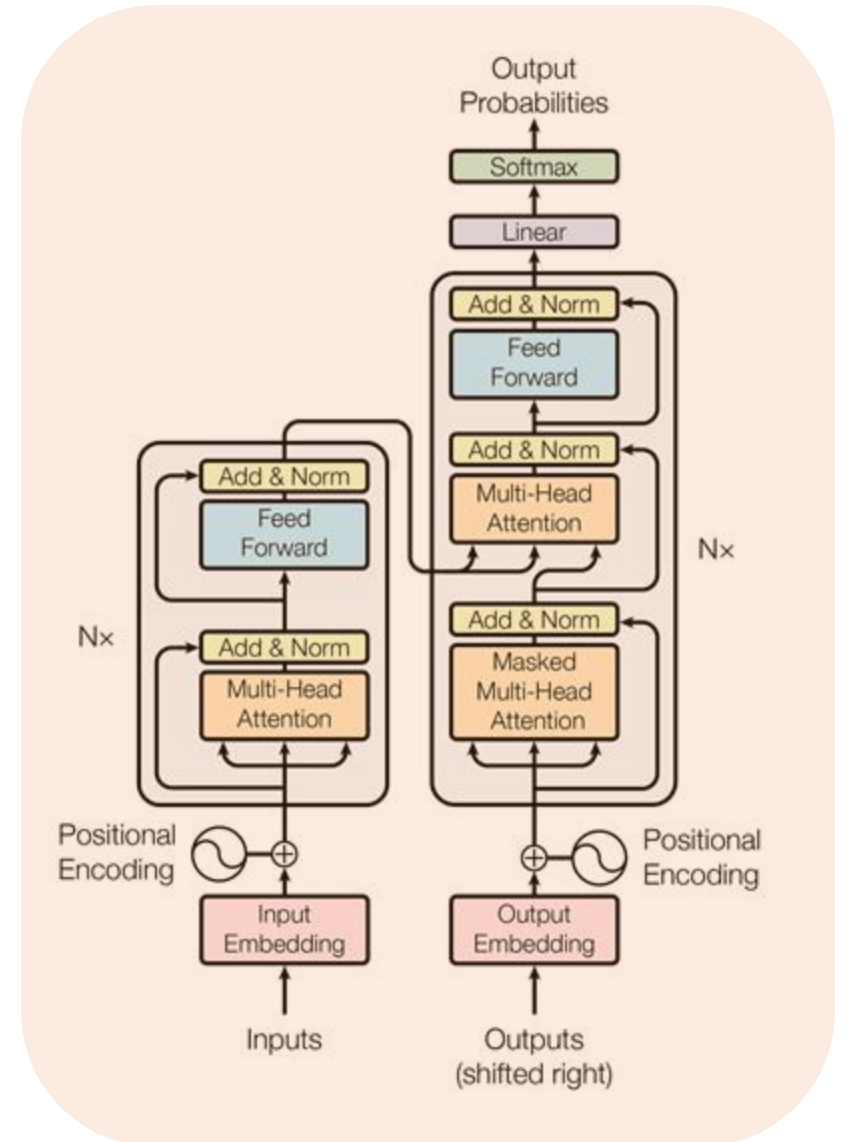
Layer Normalization

Skip Connections

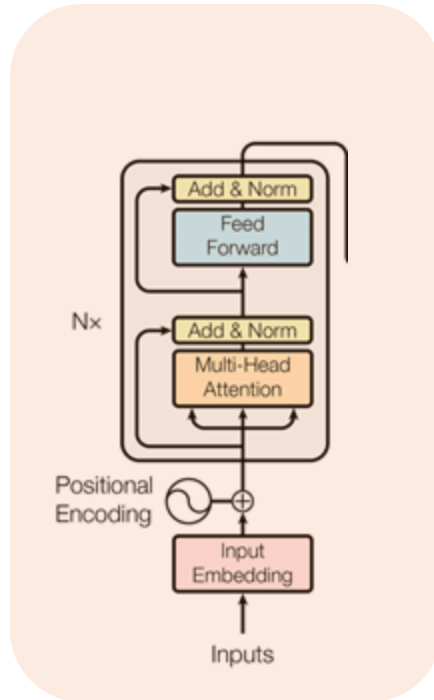
Byte Pair Encodings

Layer Stacking

Cross Attention



| Model types



Encoder

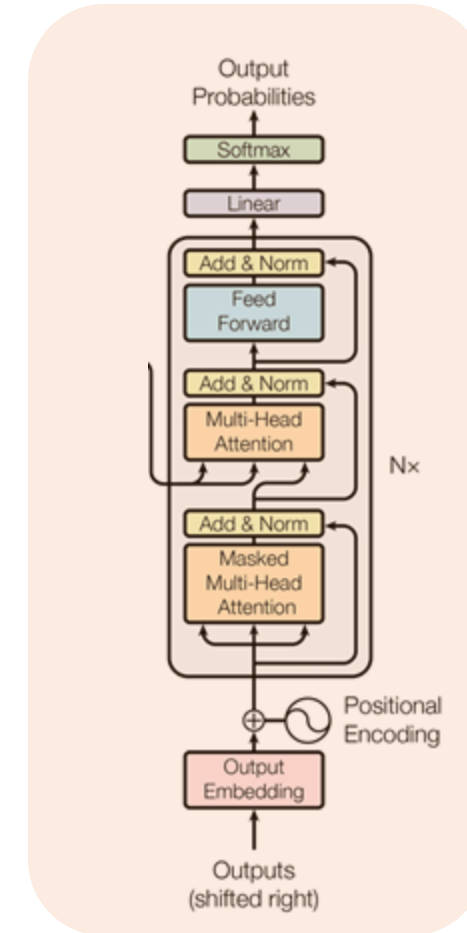
Example: BERT

Used for:
Sequence
classification

Encoder-decoder

Example: T5

Used for: Sequence-to-
sequence (e.g. machine
translation)

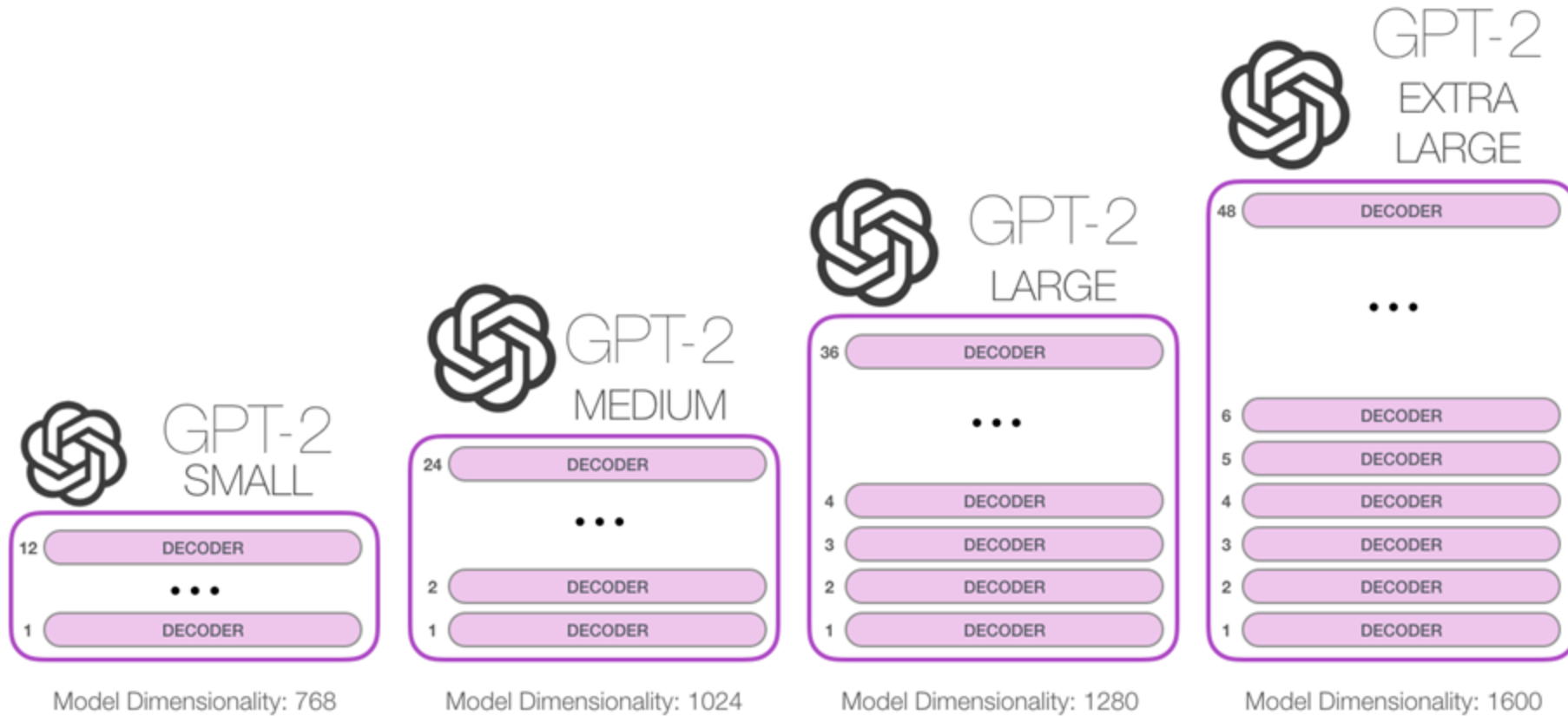


Decoder

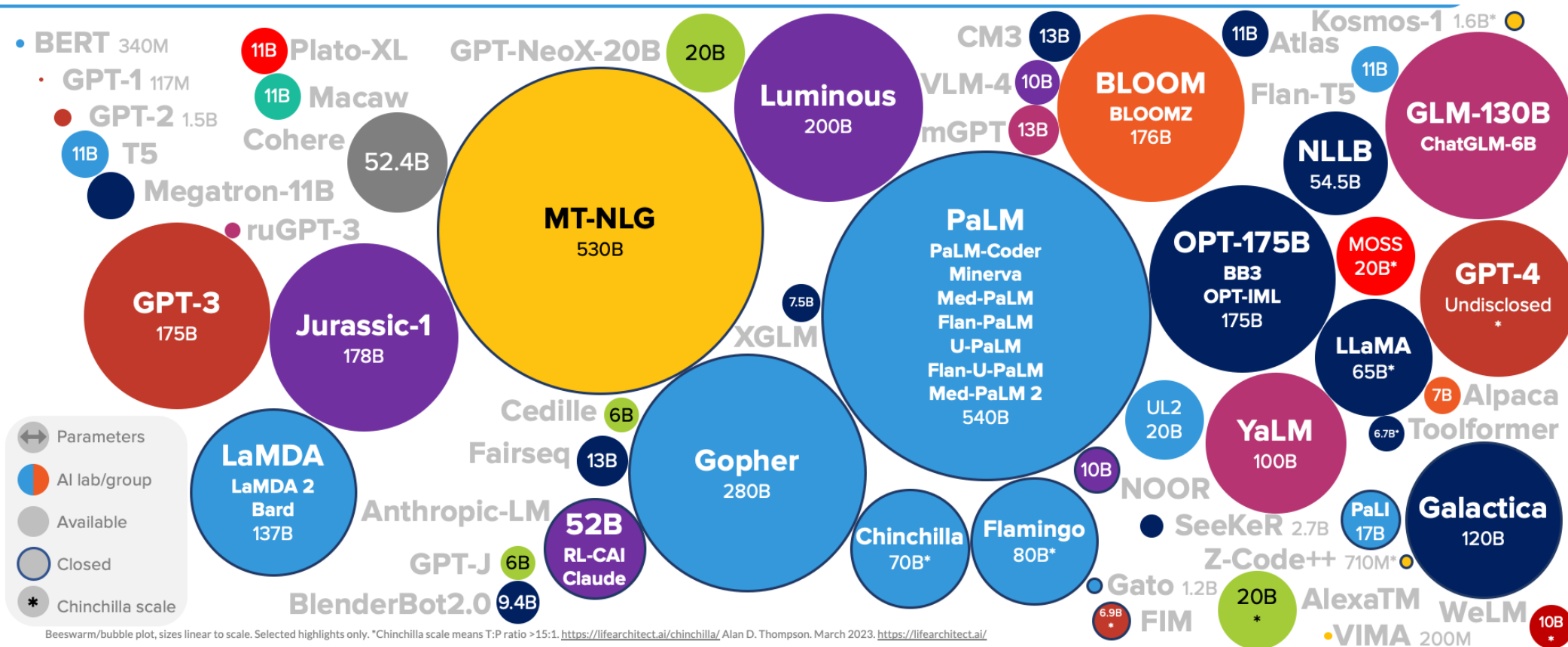
Example: GPT

Used for: Text
generation

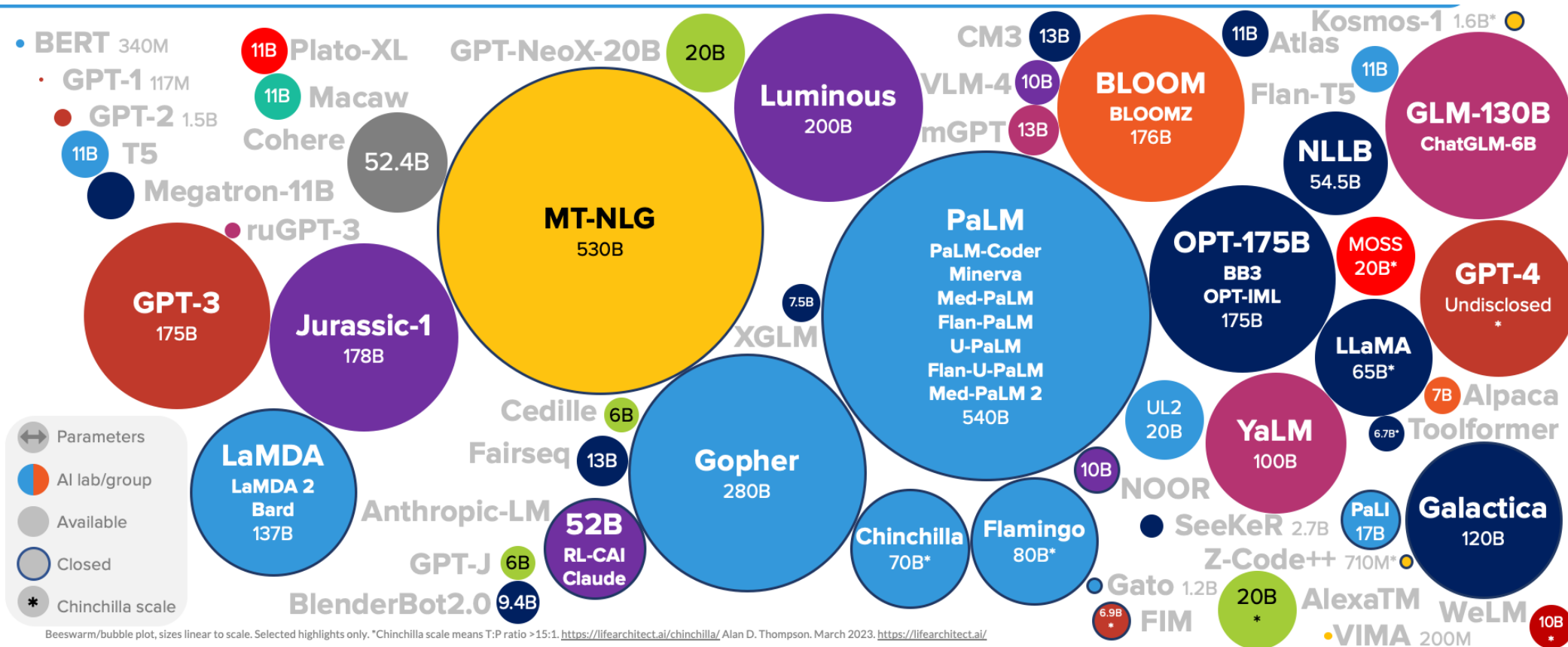
| Model types



Scaling Transformers



Scaling Transformers

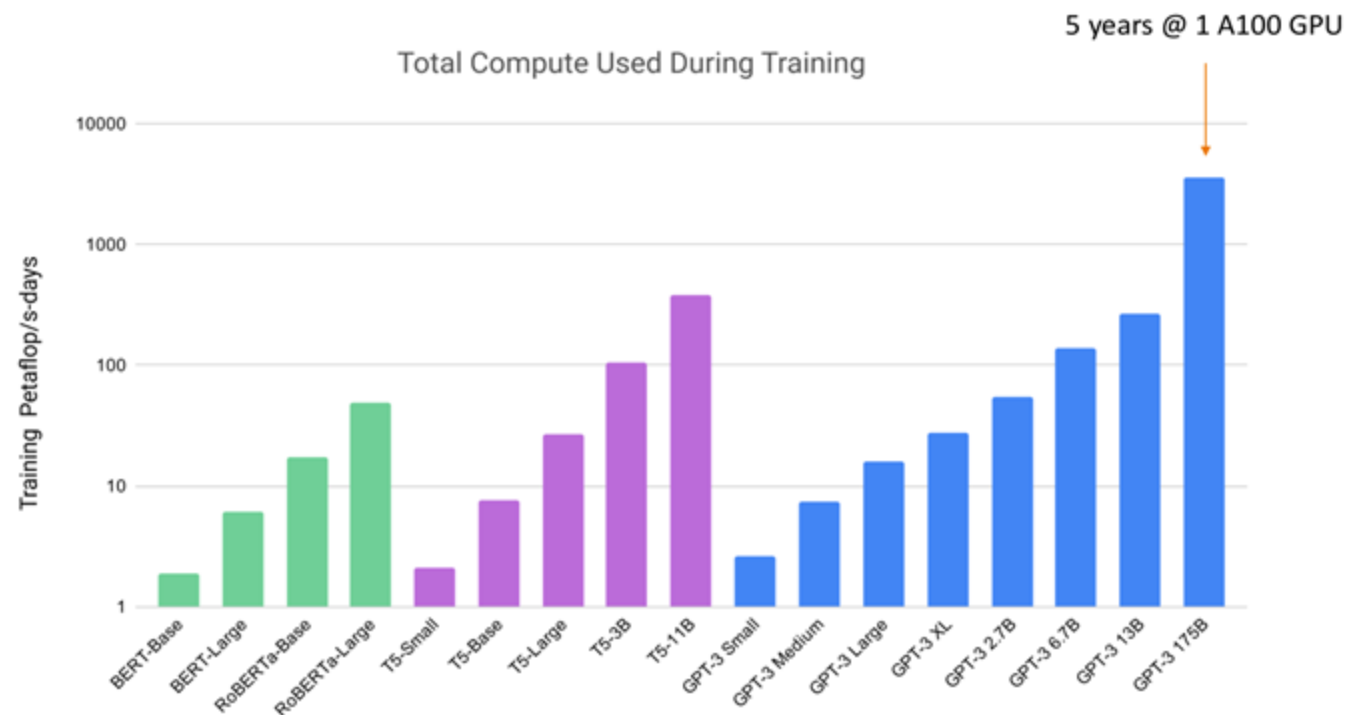


SURF

In comparison: the “big” model in original attention paper had only 213M parameters

Scaling Transformers

GPT-3: trained on ~350B tokens for 175B parameters

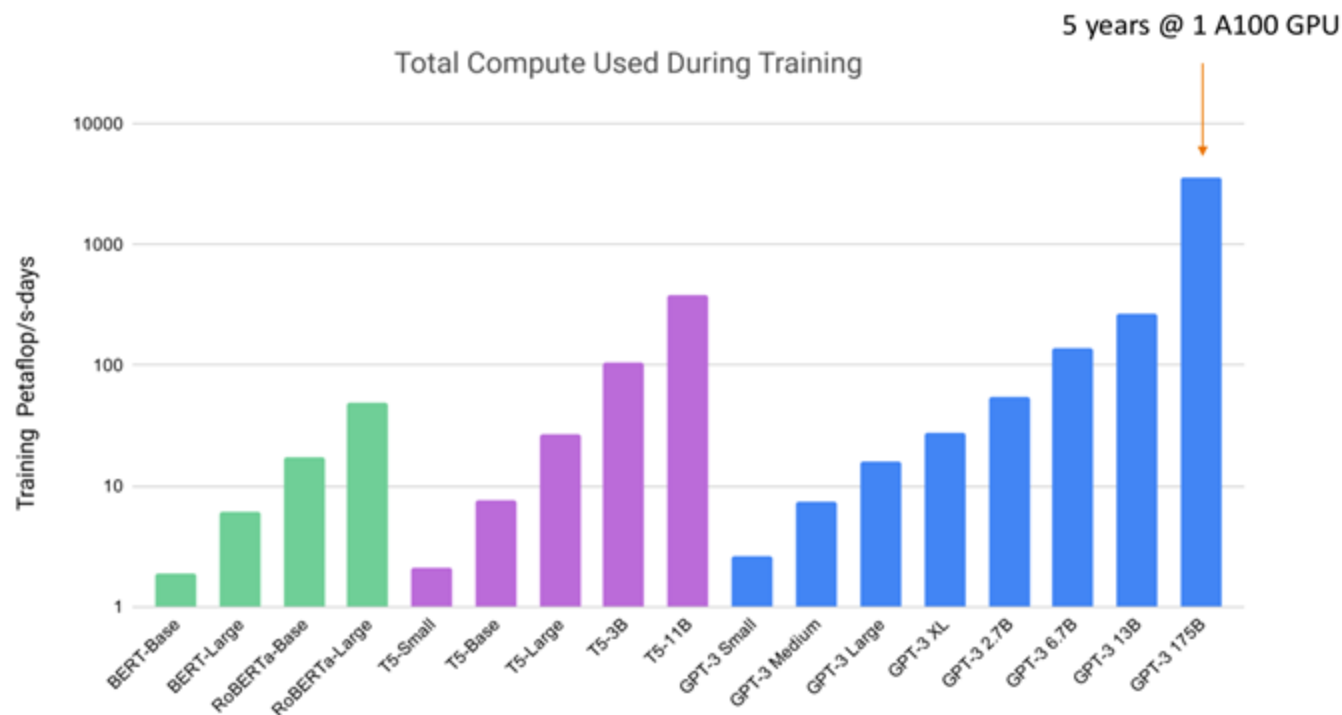


Scaling Transformers

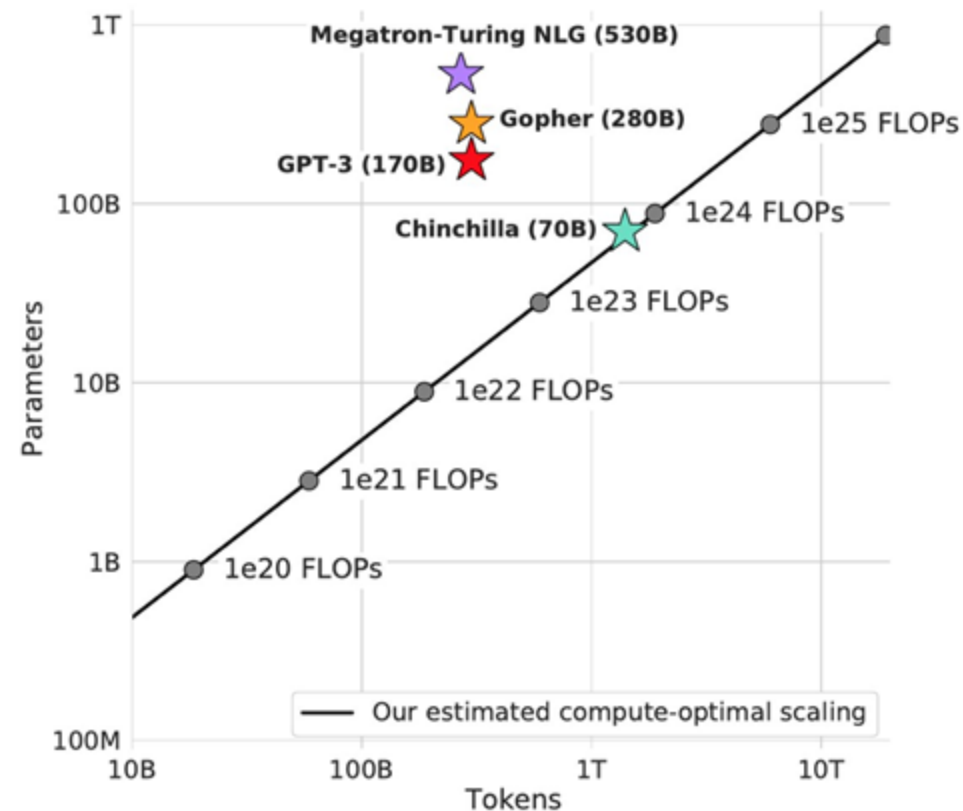
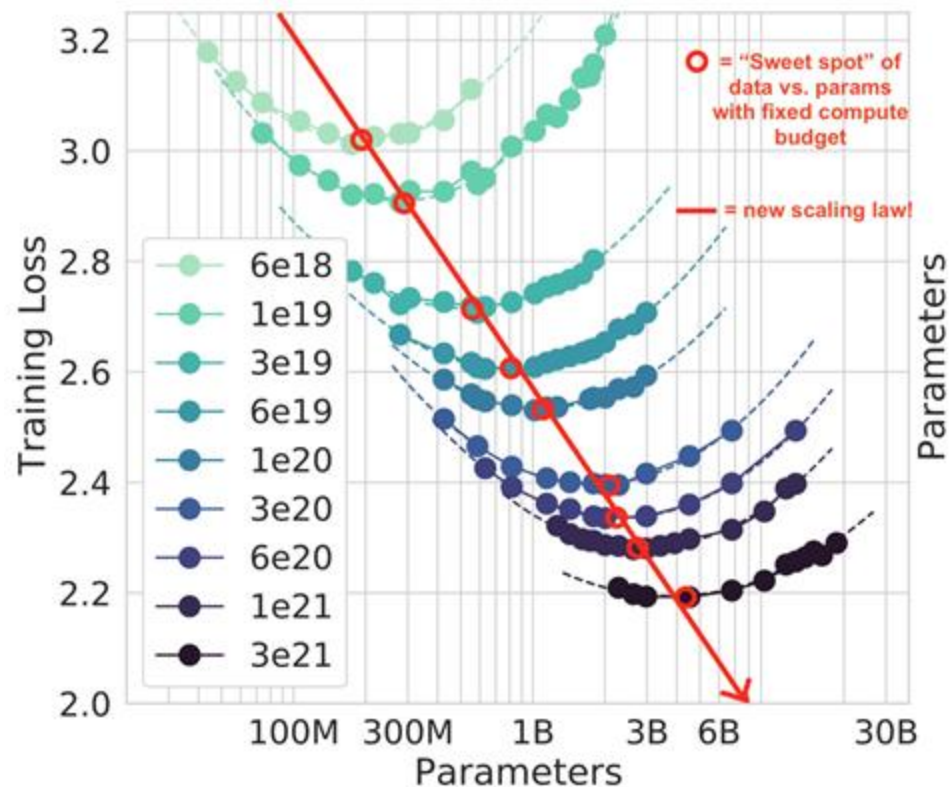
GPT-3: trained on ~350B tokens for 175B parameters

GPT4: in the range of 1.8T parameters, trained on >1T tokens.

3 months at 20.000 A100 GPUs...



Chinchilla scaling laws



Hoffmann, Jordan et al. "Training Compute-Optimal Large Language Models." *ArXiv* abs/2203.15556 (2022)

| Finetuning

Current era of LLMS: few parties with enough data and compute to train large models

What can you do without requiring 1000 H100 GPUs? 😊

| Finetuning

Current era of LLMS: few parties with enough data and compute to train large models

What can you do without requiring 1000 H100 GPUs? 😊

NO TRAINING



Use foundation models without training:

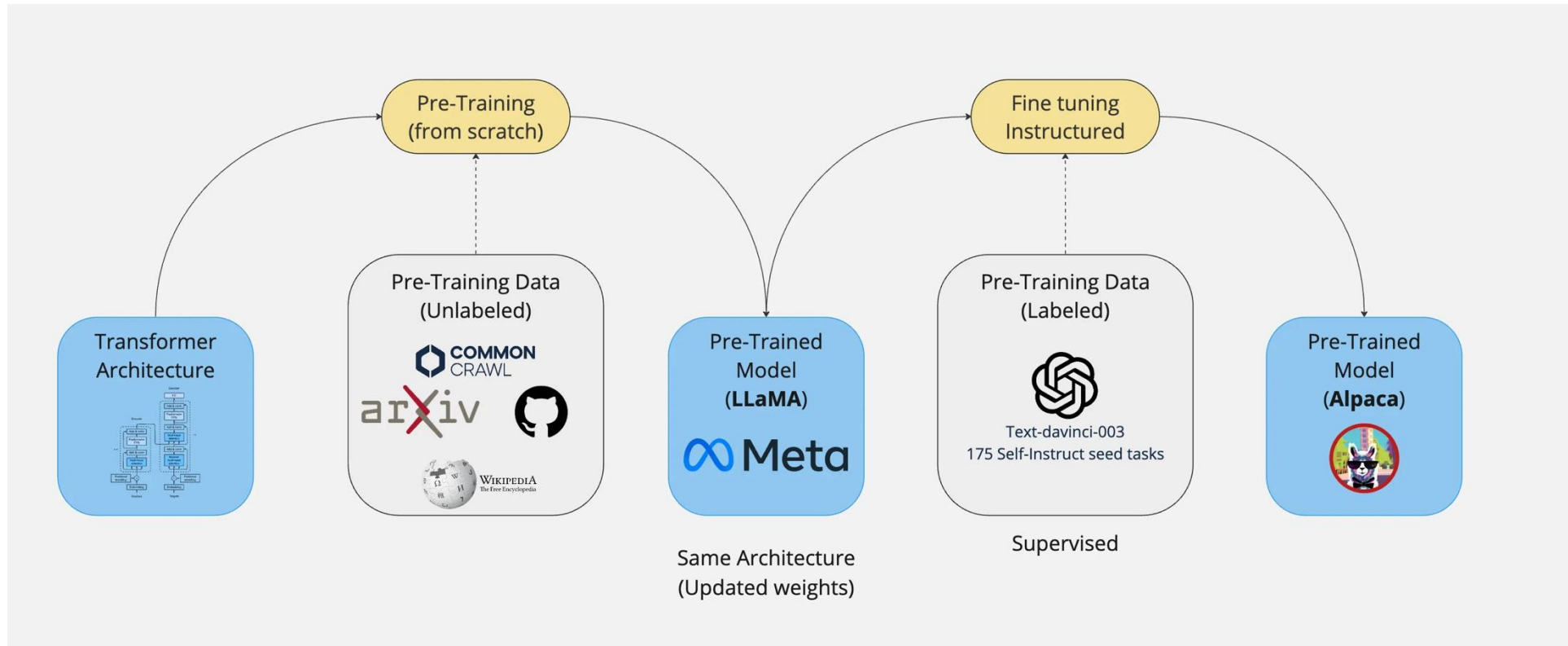
- In-context learning
- Retrieval augmented generation

FINETUNING



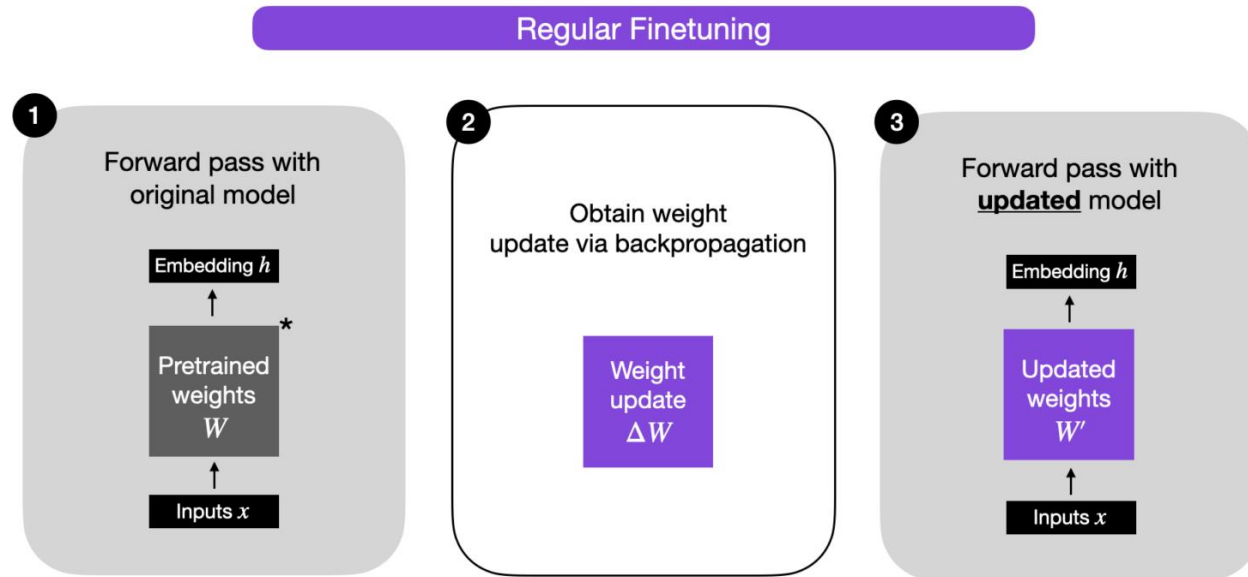
Transfer learning: finetune model to adapt to specific task

| From training to finetuning



| Efficient parameter optimization

Often not necessary to change **all** the parameters

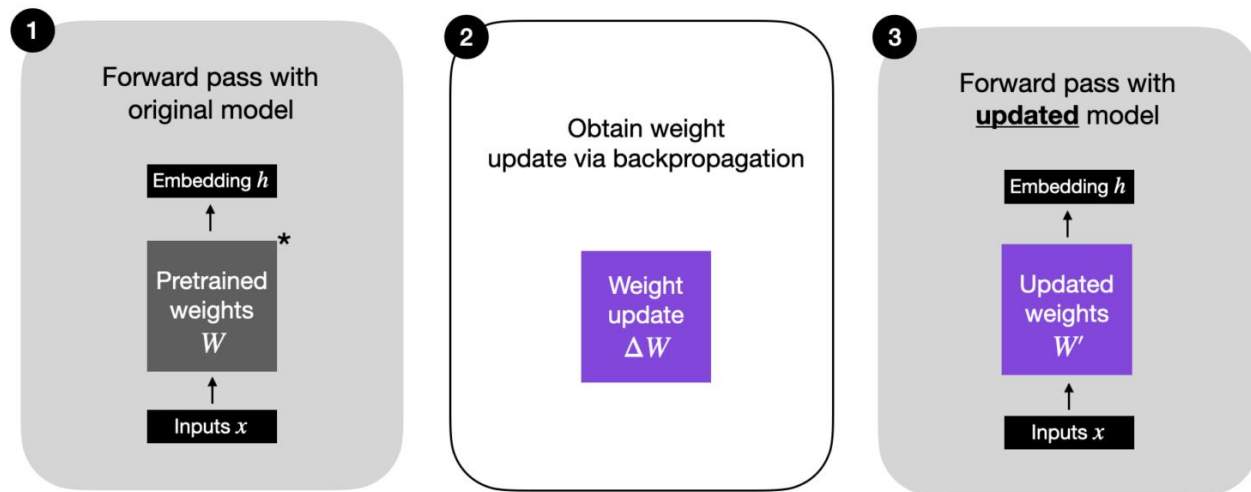


* The pretrained model could be any LLM, e.g., an encoder-style LLM (like BERT) or a generative decoder-style LLM (like GPT)

Efficient parameter optimization

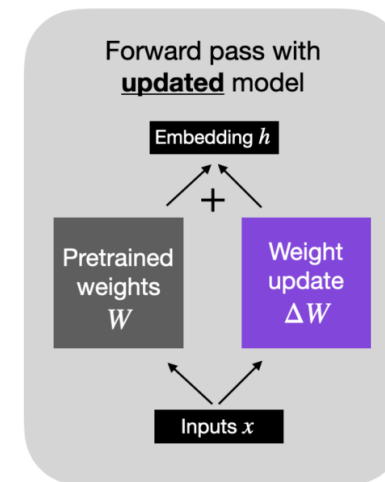
Often not necessary to change **all** the parameters

Regular Finetuning



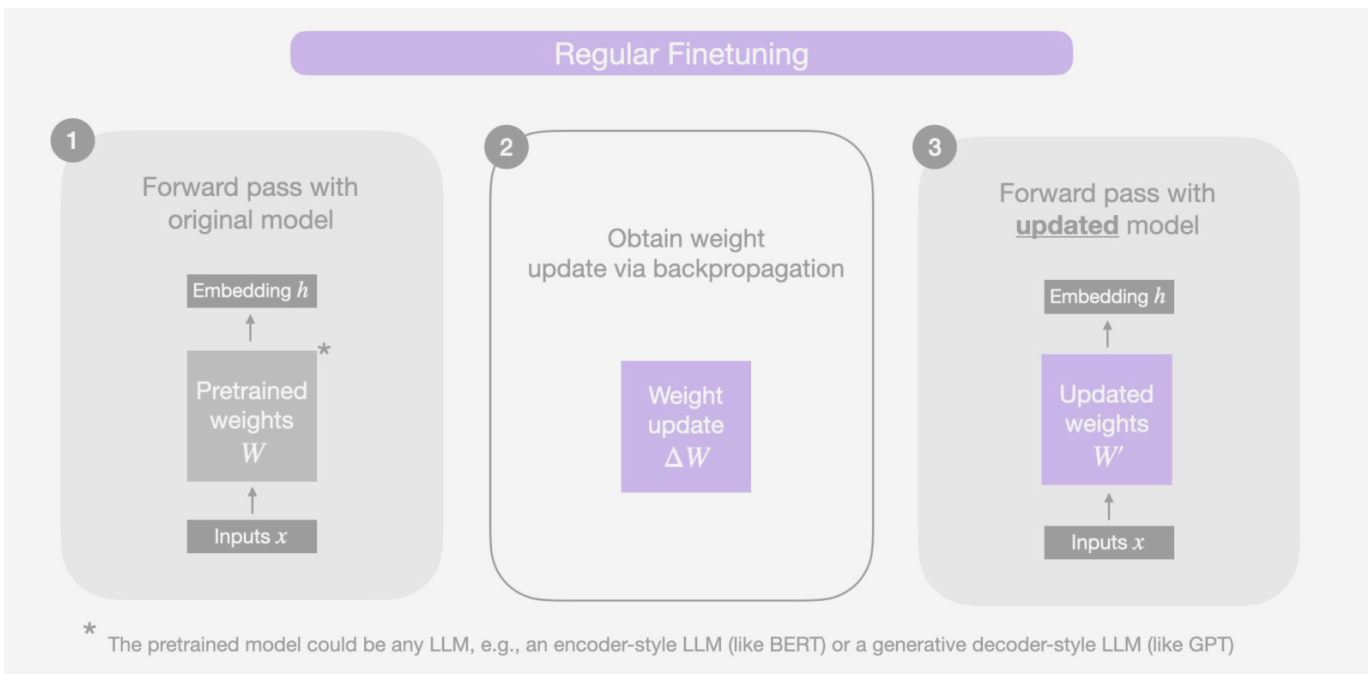
* The pretrained model could be any LLM, e.g., an encoder-style LLM (like BERT) or a generative decoder-style LLM (like GPT)

Alternative formulation (regular finetuning)



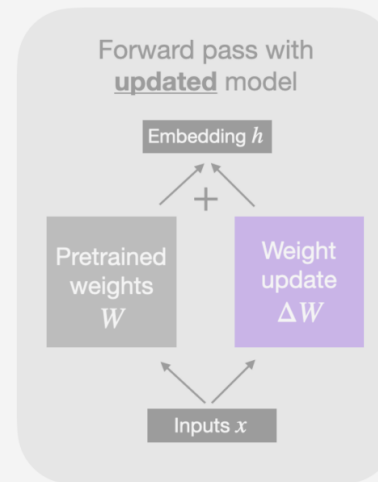
Efficient parameter optimization

Often not necessary to change **all** the parameters

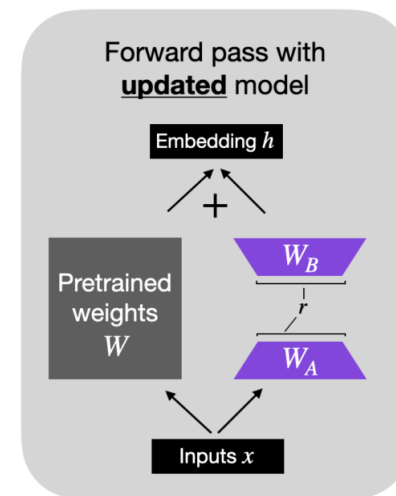


SURF

Alternative formulation (regular finetuning)



LoRA weights, W_A and W_B , represent ΔW



| Efficient parameter optimization

Often not necessary to change **all** the parameters

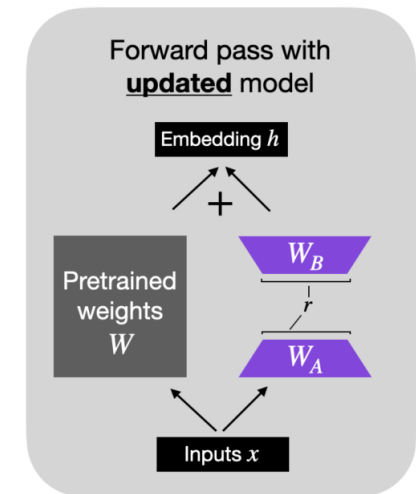
Reduces parameters in vram by about 4x!

LLama 7B in FP:

- 4 (bytes) x 7 = 28GB (inference)
- 8 (bytes) x 7 = **56GB** (training)

Using parameter-efficient adapter training, you only need **14GB**!

LoRA weights, W_A and W_B , represent ΔW



| Considerations



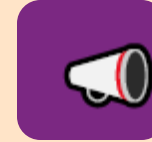
Social bias



Legal limitations



Trust and reliability



Disinformation



Environmental impact

Considerations



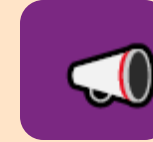
Social bias



Legal limitations



Trust and reliability



Disinformation



Environmental impact

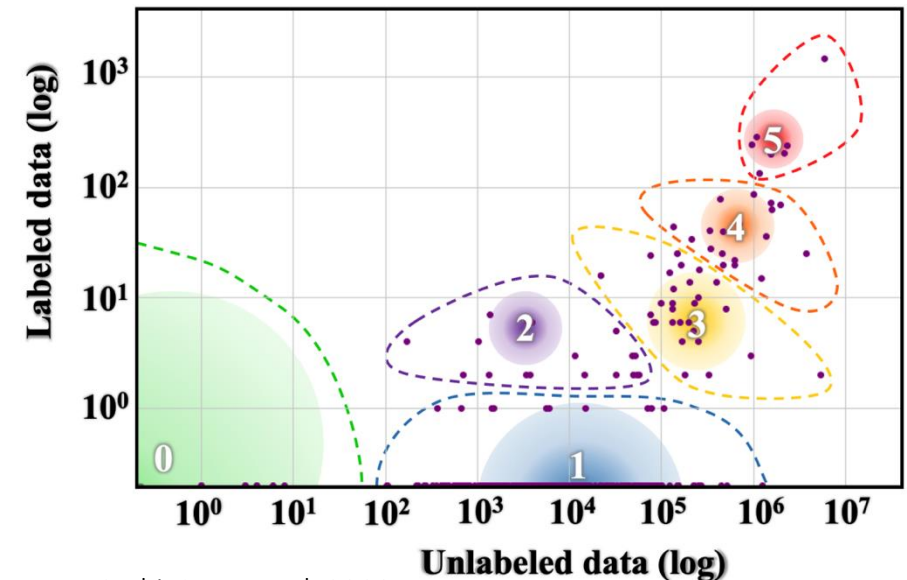
On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? 🦜

Emily M. Bender*
ebender@uw.edu
University of Washington
Seattle, WA, USA

Angelina McMillan-Major
aymm@uw.edu
University of Washington
Seattle, WA, USA

Timnit Gebru*
timnit@blackinai.org
Black in AI
Palo Alto, CA, USA

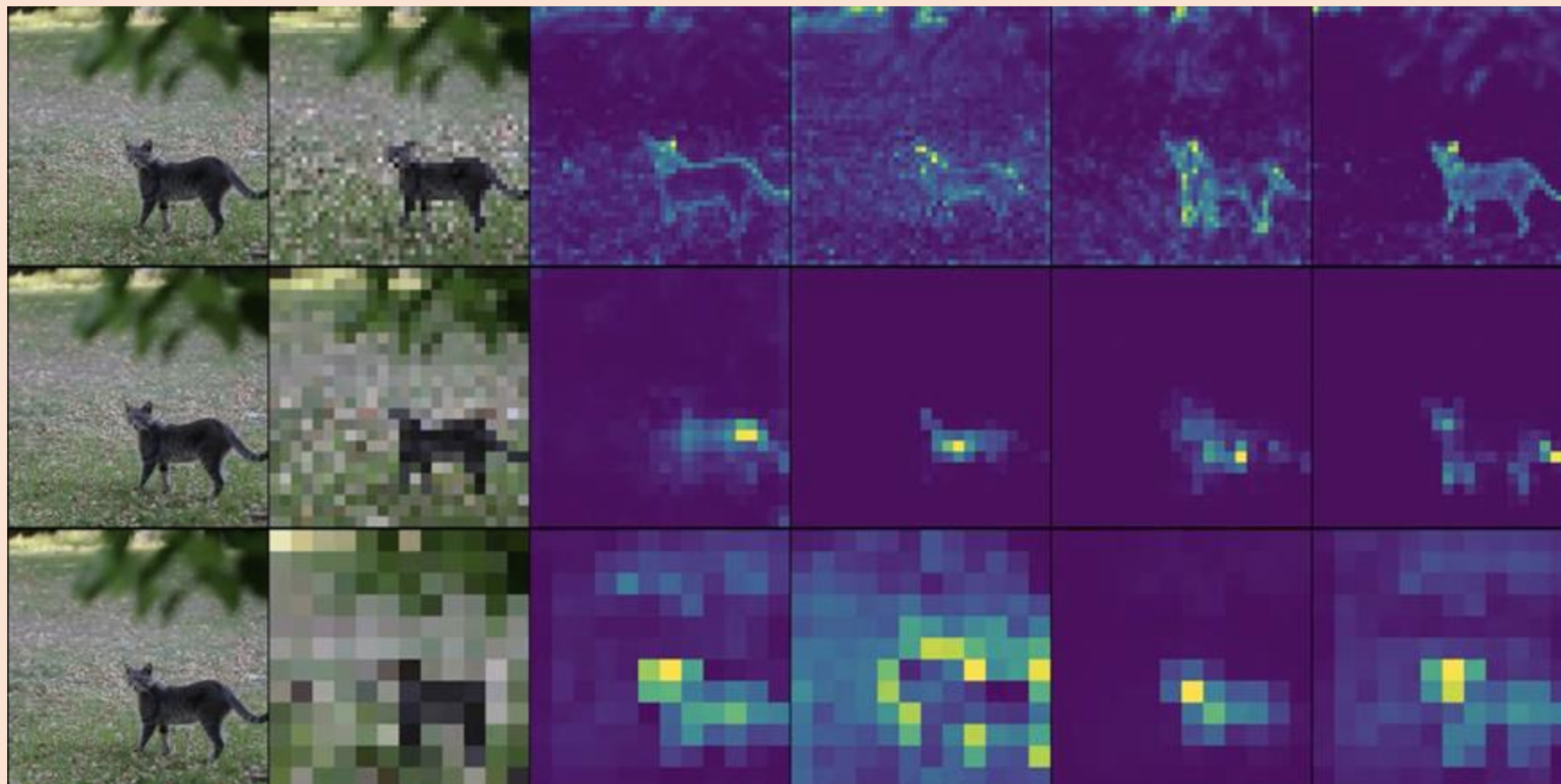
Shmargaret Shmitchell
shmargaret.shmitchell@gmail.com
The Aether



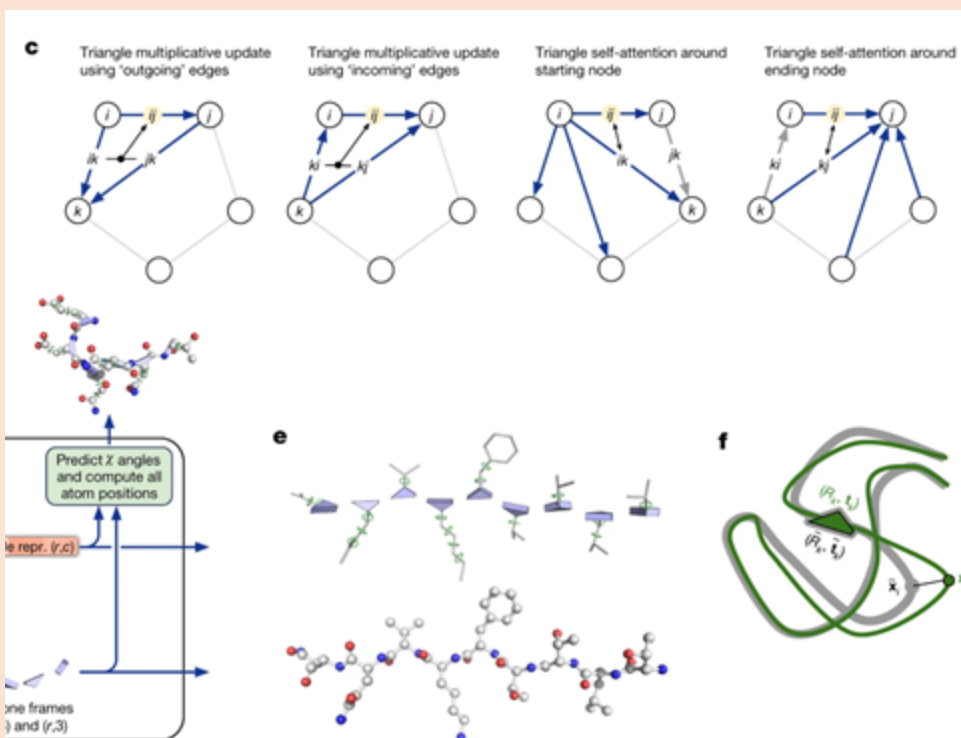
Joshi, Santy et al. 2020.

[The State and Fate of Linguistic Diversity and Inclusion in the NLP World.](#)
ACL 58.

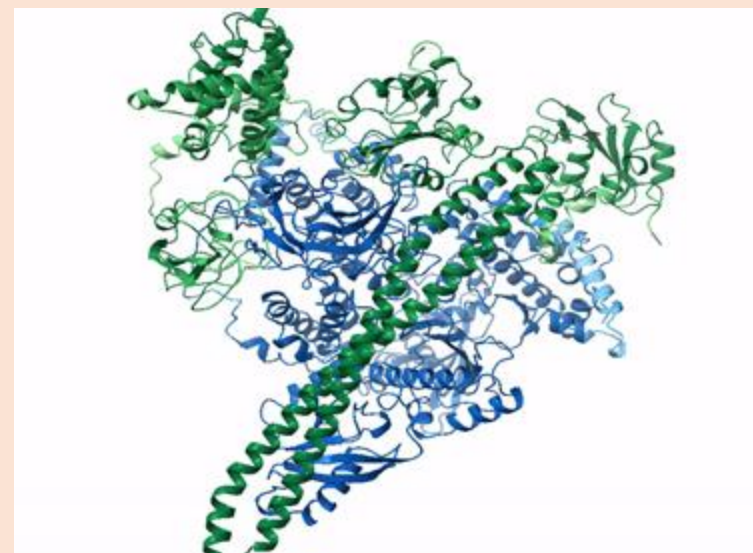
| Transformers for other modalities: ViT



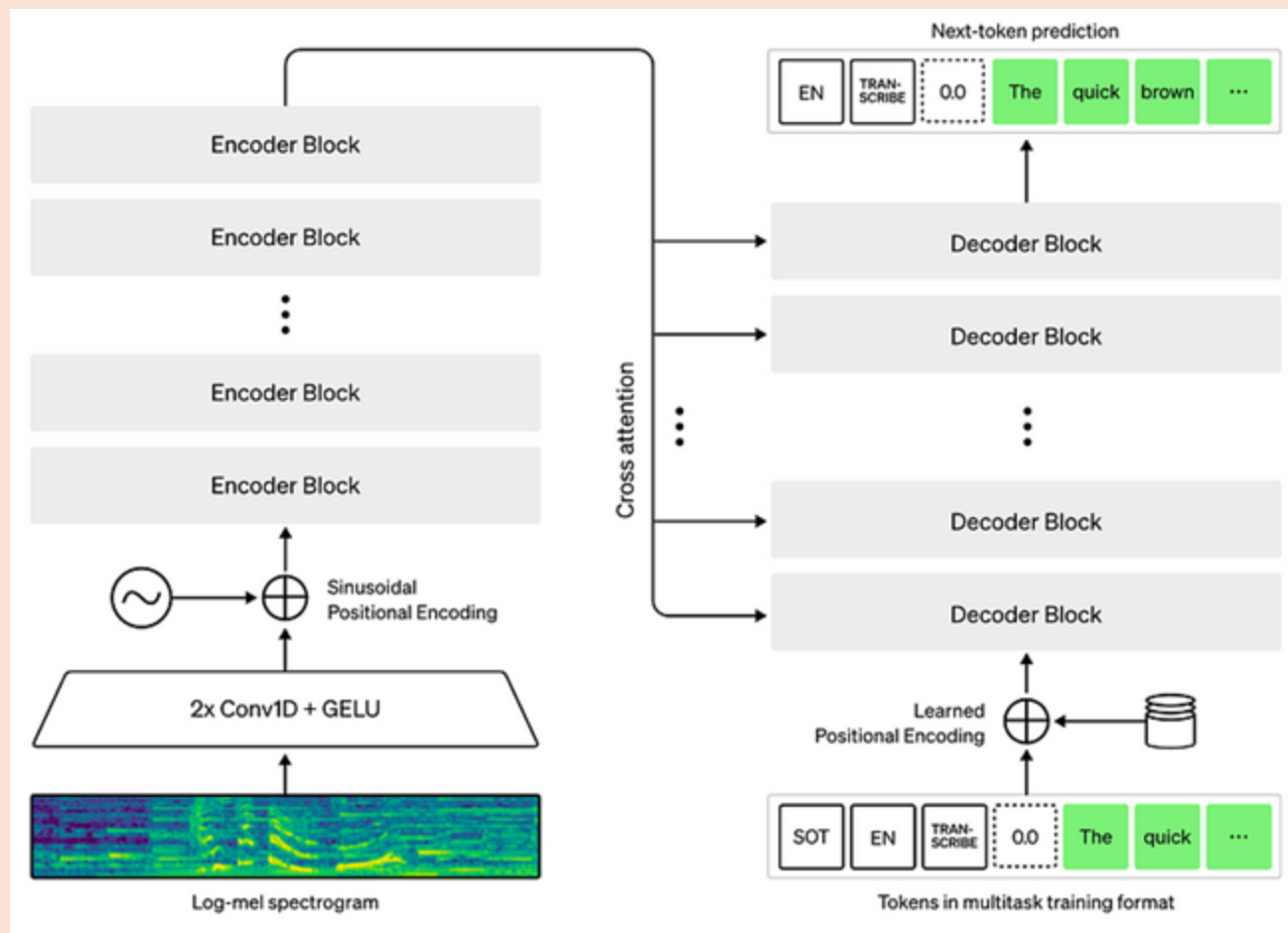
Transformers for other modalities: EvoFormer



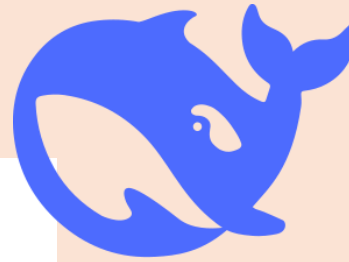
<https://www.nature.com/articles/s41586-021-03819-2>



Transformers for other modalities: Whisper



| A lot is happening...



deepseek

Motie aangenomen over AI-fabriek in Groningen

22 januari 2025

De Tweede Kamer heeft op 21 januari een motie aangenomen van Kamerlid Vermeer van BBB. De motie roept de regering op om regie te nemen over de effecten van een mogelijke AI-fabriek in Groningen op het landschap, het energiesysteem, het watergebruik en het benutten van restwarmte.

SURF



• Maandag 27-01-2025

BREIN haalt groot AI taalmodel offline

BREIN heeft afgelopen week een groot Nederlands generatief AI taalmodel offline gehaald genaamd GEITje-7B. Het model was getraind op de grootste Nederlandstalige trainingsdataset die BREIN afgelopen zomer al offline haalde ([zie: hier](#)).

| Want to learn more?

Use existing (efficient!) frameworks



Hugging Face



unsloth

Make use of SURF guides for efficient
finetuning:

<https://servicedesk.surf.nl/wiki/display/WIKI/LLM+finetuning+on+Snellius>

